

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

PROYECTO FINAL BASE DE DATOS I

SISTEMA DE VENTAS - EDEN STORE

Autor (es):
ARANDY ESDUARDO COLLADO ZAPETA 1090-23-5761

Profesor
MARCOS URIZAR

Curso
BASE DE DATOS I

Guatemala, Octubre 2026

Índice

Introducción 4
1. Análisis de requerimientos 4
1.1 Descripción del problema 4
1.2 Objetivos 5
1.3 Alcance 5
1.4 Actores 6
1.5 Requerimientos funcionales 6
1.6 Requerimientos no funcionales 7
1.7 Reglas del negocio 7
1.8 Correspondencia entre requerimientos e implementación 9
2. Diseño de la base de datos 11
2.1 Entidades y atributos 11
2.2 Relaciones y cardinalidades 13
2.3 Resolución de relaciones de muchos a muchos 15
2.4 Diagrama Entidad–Relación 15
2.5 Modelo relacional 22
2.6 Modelo físico 25
3. Normalización 28
3.1 Relación sin normalizar (0FN) 28
3.2 Primera forma normal (1FN) 28
3.3 Segunda forma normal (2FN) 29
3.4 Tercera forma normal (3FN) 30
3.5 Decisiones de diseño sobre datos calculados 31
4. Implementación en Oracle 31
4.1 Entorno 31
4.2 Organización del script 32
4.3 Secuencias e índices 32
4.4 Vistas 33
4.5 Funciones 33
4.6 Procedimientos almacenados 34

4.7 Triggers 36
4.8 Tarea programada 37
4.9 Transacciones: COMMIT y ROLLBACK 37
4.10 Manejo de excepciones 37
5. Consultas SQL y explotación de la información 38
6. Seguridad 39
6.1 Seguridad en Oracle 39
6.2 Seguridad en la aplicación 40
7. Prototipo funcional 41
7.1 Arquitectura 41
7.2 Pantallas principales 41
8. Pruebas y validación 49
8.1 Datos de prueba 49
8.2 Verificación de la consistencia de los datos 49
8.3 Pruebas de reglas de negocio y manejo de errores 50
8.4 Validaciones en la aplicación 51
9. Instalación, ejecución y respaldo 51
9.1 Base de datos 51
9.2 Aplicación 51
9.3 Respaldo y restauración 51
10. Conclusiones y recomendaciones 52
10.1 Conclusiones 52
10.2 Recomendaciones y mejoras futuras 53
Anexo A. Diccionario de datos 54

Introducción

Este documento describe el análisis, el diseño, la implementación y las pruebas del **Sistema de Ventas de Eden Store**, un negocio guatemalteco que vende equipo de cómputo y accesorios. El sistema reemplaza el control manual en hojas de cálculo por una base de datos **Oracle Database 21c XE** normalizada hasta la tercera forma normal, con reglas de negocio garantizadas por restricciones y bloques PL/SQL, seguridad por roles y un prototipo web en **Python (Flask)** conectado directamente a Oracle.

El documento sigue el orden de la lista de cotejo del proyecto: análisis de requerimientos (sección 1), diseño de la base de datos (2), normalización (3), implementación en Oracle (4), consultas SQL (5), seguridad (6), prototipo funcional (7), pruebas (8), instalación y respaldo (9) y conclusiones (10). Al final se incluye el diccionario de datos completo como anexo.

Todos los objetos se crean con un único script, database/proyecto_sistema_ventas.sql, y el código fuente se encuentra en el repositorio del proyecto.

1. Análisis de requerimientos

1.1 Descripción del problema

Eden Store registra sus compras, ventas e inventario en hojas de cálculo y documentos separados. Esta forma de trabajo provoca los siguientes problemas:

- **Inventario descuadrado:** las existencias reales no coinciden con las registradas, porque cada venta o compra se anota a mano y en archivos distintos.
- **Ventas sin existencia:** se ofrecen y venden productos que ya no hay en bodega.
- **Falta de trazabilidad:** no se sabe quién ni cuándo cambió una existencia o un precio.
- **Cotizaciones confundidas con ventas:** una cotización se anota igual que una venta y descuenta producto.
- **Caja sin control formal:** los faltantes y sobrantes no tienen responsable ni seguimiento.
- **Anulaciones sin autorización:** cualquier persona puede borrar una venta sin dejar registro.

- **Acceso sin control:** todos los empleados pueden ver y modificar toda la información.

1.2 Objetivos

Objetivo general

Desarrollar un Sistema de Ventas con una base de datos Oracle normalizada e íntegra que permita registrar compras, ventas, cotizaciones, pagos y movimientos de caja, mantener el inventario actualizado y trazable, y controlar el acceso según el rol de cada usuario.

Objetivos específicos

1. Diseñar una base de datos relacional normalizada hasta la tercera forma normal (3FN) con integridad referencial.
2. Automatizar en Oracle (procedimientos, funciones, triggers y jobs) la actualización del inventario, el Kardex y la caja.
3. Garantizar con transacciones que ninguna operación quede registrada a medias.
4. Controlar la caja con aperturas, cierres, faltantes, sobrantes, vales de descuento y sanciones sugeridas.
5. Implementar seguridad por roles tanto en Oracle como en la aplicación.
6. Desarrollar un prototipo web conectado a Oracle con reportes para la toma de decisiones.

1.3 Alcance

El sistema incluye: inicio de sesión y usuarios con uno o varios roles; gestión de clientes, proveedores y productos; compras y ventas con varios productos; pagos en efectivo, tarjeta, transferencia o combinados; anulación de ventas con autorización de un administrador; recibo imprimible; cotizaciones con vigencia y conversión a venta; Kardex; apertura y cierre de caja; faltantes, sobrantes, vales y sanciones sugeridas; auditoría de precios; reportes; tarea programada; índices y respaldos.

No incluye (mejoras futuras): facturación electrónica FEL (SAT); notas de crédito para ventas de cajas ya cerradas; devoluciones parciales; créditos y cuentas por cobrar; contabilidad; varias sucursales o bodegas.

1.4 Actores

Actor	Tipo	Responsabilidades
Administrador	Interno	Usuarios, cajas, precios, autorización de anulaciones, reportes, incidencias y vales.
Vendedor	Interno	Clientes, cotizaciones, ventas y su propia caja.
Cajero	Interno	Apertura y cierre de caja, ventas y solicitud de anulaciones.
Bodeguero	Interno	Proveedores, productos, compras y Kardex.
Consultor de productos	Interno (solo BD)	Consulta del catálogo de productos directamente en Oracle.
Cliente	Externo	Solicita cotizaciones y compra; no usa el sistema.
Proveedor	Externo	Abastece productos; no usa el sistema.

1.5 Requerimientos funcionales

ID	Requerimiento
RF-01	Iniciar sesión validando la contraseña y que el usuario esté activo.
RF-02	Mostrar solo los módulos permitidos para el rol del usuario.
RF-03	Crear, modificar, activar y desactivar usuarios con uno o varios roles.
RF-04	Registrar, modificar y eliminar clientes.
RF-05	Registrar, modificar y eliminar proveedores.
RF-06	Registrar, modificar, activar y desactivar productos.
RF-07	Consultar existencias e identificar los productos con bajo stock.
RF-08	Auditar automáticamente los cambios de precio de venta.
RF-09	Registrar compras con varios productos, actualizando existencias y Kardex.
RF-10	Registrar ventas con varios productos validando cliente, usuario, caja abierta, producto activo y stock.
RF-11	Registrar uno o varios pagos por venta, con cálculo del cambio.
RF-12	Imprimir o guardar en PDF el comprobante de venta.
RF-13	Anular ventas con autorización de un administrador, devolviendo el stock.
RF-14	Elaborar cotizaciones con fecha de vencimiento sin afectar el inventario.
RF-15	Caducar automáticamente las cotizaciones vencidas.
RF-16	Convertir una cotización abierta en venta.
RF-17	Abrir y cerrar caja calculando el monto esperado y la diferencia.

ID	Requerimiento
RF-18	Generar automáticamente un vale de descuento cuando hay faltante en el cierre.
RF-19	Consultar incidencias, aplicar o anular vales y ver sanciones sugeridas.
RF-20	Consultar el Kardex por producto y tipo de movimiento.
RF-21	Generar reportes de ventas, inventario, clientes, usuarios y métodos de pago.
RF-22	Respaldar y restaurar la base de datos.

1.6 Requerimientos no funcionales

ID	Categoría	Requerimiento
RNF-01	Seguridad	Contraseñas almacenadas con hash (scrypt); acceso por roles en Oracle y en la aplicación.
RNF-02	Integridad	Compras, ventas, pagos y anulaciones son transacciones atómicas.
RNF-03	Consistencia	Las reglas críticas se validan en la base de datos (restricciones y PL/SQL).
RNF-04	Usabilidad	Interfaz web minimalista y mensajes de error comprensibles.
RNF-05	Rendimiento	Índices en todas las llaves foráneas y en las consultas frecuentes.
RNF-06	Disponibilidad	Respaldos con Oracle Data Pump.
RNF-07	Portabilidad	Ejecución local y posible despliegue en Ubuntu Server.
RNF-08	Mantenibilidad	Patrón Modelo–Vista–Controlador (MVC).

1.7 Reglas del negocio

ID	Regla	Dónde se garantiza
RN-01	Un usuario inactivo no inicia sesión, no abre caja ni vende.	Login, SP_ABRIR_CAJA, SP_CREAR_VENTA
RN-02	Un usuario puede tener varios roles.	Tabla asociativa USUARIO_ROL

ID	Regla	Dónde se garantiza
RN-03	No se repiten el nombre de usuario, el correo de un cliente ni el NIT de un proveedor.	Restricciones UNIQUE
RN-04	El stock nunca es negativo.	CHECK (stock >= 0) y validación en PL/SQL
RN-05	El stock solo cambia por compras, ventas y anulaciones.	Procedimientos almacenados
RN-06	Todo movimiento de inventario queda en el Kardex.	Procedimientos almacenados
RN-07	El precio de venta es mayor o igual al precio de compra.	CHECK chk_productos_precios
RN-08	Productos y usuarios con historial se desactivan, no se eliminan.	Columna estado y llaves foráneas
RN-09	Un producto inactivo no se compra ni se vende.	Procedimientos almacenados
RN-10	Para vender se necesita una caja abierta; toda venta pertenece a una apertura.	SP_CREAR_VENTA, id_apertura NOT NULL
RN-11	El precio de venta lo toma el sistema del catálogo, no el usuario.	SP_AGREGAR_DETALLE_VENTA
RN-12	La suma de los pagos debe ser igual al total de la venta.	SP_FINALIZAR_VENTA
RN-13	Tarjeta y transferencia requieren referencia; solo el efectivo registra monto recibido.	SP_AGREGAR_PAGO_VENTA y CHECK
RN-14	Solo el efectivo de ventas activas suma al monto esperado de la caja.	FN_EFECTIVO_APERTURA
RN-15	Una cotización no modifica el inventario.	Módulo de cotizaciones
RN-16	Una cotización vencida pasa a Caducada; solo una abierta se factura y una sola vez.	SP_CADUCAR_COTIZACIONES, UNIQUE (id_cotizacion)
RN-17	Un usuario solo tiene una caja abierta y una caja solo la abre un usuario a la vez.	SP_ABRIR_CAJA
RN-18	Solo quien abrió la caja puede cerrarla.	SP_CERRAR_CAJA
RN-19	`Diferencia = monto contado − monto esperado.`	SP_CERRAR_CAJA

ID	Regla	Dónde se garantiza
RN-20	Un faltante genera un vale de descuento; un sobrante solo se registra como incidencia.	TRG_VALE_FALTANTE, VW_INCIDENCIAS_CAJA
RN-21	Un vale aplicado requiere fecha y un vale anulado requiere motivo.	CHECK de dos columnas
RN-22	Solo se anula una venta si su caja sigue abierta, con motivo y autorización de un administrador.	SP_ANULAR_VENTA
RN-23	Una venta anulada conserva fecha, solicitante, autorizador y motivo.	CHECK chk_ventas_anulacion
RN-24	La sanción se sugiere según faltantes y anulaciones del mes; decide el administrador.	POLITICA_SANCION, VW_SANCIONES_MES
RN-25	Todo cambio de precio queda auditado.	TRG_AUDITORIA_PRECIO
RN-26	No se elimina un cliente o proveedor que tenga operaciones.	Llaves foráneas

1.8 Correspondencia entre requerimientos e implementación

La siguiente tabla relaciona cada requerimiento funcional con el objeto de Oracle que lo resuelve, la pantalla de la aplicación donde se usa y la evidencia que demuestra que funciona (consultas de la sección 11 y pruebas de la sección 12 del script, o figuras de este documento).

ID	Objeto en Oracle	Pantalla (ruta)	Evidencia
RF-01	USUARIO, USUARIO_ROL	Inicio de sesión (/login)	Usuario inactivo rechazado; Figura 8
RF-02	USUARIO_ROL + permisos de la app	Menú lateral	Menú distinto por rol; Figura 9
RF-03	USUARIO, ROL, USUARIO_ROL	Usuarios (/usuarios)	Alta de usuario con varios roles
RF-04	CLIENTES, SP_REGISTRAR_CLIENTE	Clientes (/clientes)	Prueba 12.11
RF-05	PROVEEDORES	Proveedores (/proveedores)	Prueba 12.13

ID	Objeto en Oracle	Pantalla (ruta)	Evidencia
RF-06	PRODUCTOS	Productos (/productos)	Prueba 12.10; Figura 10
RF-07	FN_STOCK_PRODUCTO, VW_PRODUCTOS_BAJO_STOCK	Productos, filtro «Bajo stock»; Inicio	Figuras 9 y 10
RF-08	TRG_AUDITORIA_PRECIO, AUDITORIA_PRODUCTO	Auditoría de precios	Figura 17
RF-09	SP_AGREGAR_DETALLE_COMPRA	Compras (/compras/nueva)	Consulta 11.2
RF-10	SP_CREAR_VENTA, SP_AGREGAR_DETALLE_VENTA	Ventas (/ventas/nueva)	Pruebas 12.1, 12.2 y 12.4
RF-11	SP_AGREGAR_PAGO_VENTA, SP_FINALIZAR_VENTA	Ventas (/ventas/nueva)	Pruebas 12.5 y 12.6; consulta 11.4
RF-12	VENTAS, DETALLE_VENTA, PAGO_VENTA	Recibo (/ventas/recibo/<id>)	Figura 12
RF-13	SP_ANULAR_VENTA	Detalle de venta	Pruebas 12.7 y 12.8; Figura 11
RF-14	COTIZACION, DETALLE_COTIZACION	Cotizaciones (/cotizaciones)	Figura 18
RF-15	SP_CADUCAR_COTIZACIONES, JOB_CADUCAR_COTIZACIONES	Automático (todos los días 00:05)	Consulta 11.13
RF-16	SP_CREAR_VENTA (parámetro de cotización)	Cotizaciones → Convertir en venta	Cotización #2 facturada
RF-17	SP_ABRIR_CAJA, SP_CERRAR_CAJA	Caja (/caja)	Consulta 11.5; Figura 15
RF-18	TRG_VALE_FALTANTE	Caja → Vales	Vale #3; Figura 16

ID	Objeto en Oracle	Pantalla (ruta)	Evidencia
RF-19	VW_INCIDENCIAS_CAJA, VW_SANCIONES_MES, SP_APLICAR_VALES_MES	Caja → Incidencias y Vales	Figuras 14 y 16
RF-20	KARDEX	Kardex (/kardex)	Consulta 11.2; Figura 13
RF-21	Vistas y consultas de agregación	Reportes (/reportes)	Figura 19
RF-22	Oracle Data Pump (expdp / impdp)	Línea de comandos	Sección 9.3

2. Diseño de la base de datos

La base de datos tiene **20 tablas** agrupadas en siete módulos. El diseño parte de las entidades que aparecen en el análisis (personas, productos, documentos y caja) y resuelve todas las relaciones de muchos a muchos con entidades asociativas.

2.1 Entidades y atributos

Módulo	Entidad	Descripción	Atributos principales
Seguridad	ROL	Perfiles de acceso (Administrador, Vendedor, Cajero, Bodeguero).	id_rol, nombre_rol, descripcion
	USUARIO	Personas que usan el sistema.	id_usuario, usuario, password (hash), estado
	USUARIO_ROL	Asociativa: roles de cada usuario.	id_usuario, id_rol
Personas	CLIENTES	Compradores de la tienda.	id_cliente, nombre, direccion, telefono, correo
	PROVEEDORES	Empresas que abastecen productos.	id_proveedor, nombre, direccion, telefono, nit
Inventario	PRODUCTOS	Catálogo con precios y existencias.	id_producto, nombre, precio_compra, precio_venta, stock, stock_minimo, estado

Módulo	Entidad	Descripción	Atributos principales
	KARDEX	Historial de cada entrada y salida de inventario.	id_kardex, id_producto, fecha, tipo_movimiento, referencia, cantidad_entrada, cantidad_salida, existencia_final
	AUDITORIA_PRODUCTO	Bitácora de cambios de precio.	id_auditoria, id_producto, precio_anterior, precio_nuevo, usuario, fecha
Compras	COMPRAS	Encabezado de una compra a un proveedor.	id_compra, fecha, total, id_proveedor, id_usuario
	DETALLE_COMPRA	Productos de cada compra.	id_detalle_compra, id_compra, id_producto, cantidad, precio_costo, subtotal
Ventas	VENTAS	Encabezado de una venta, con datos de anulación.	id_venta, fecha, total, id_cliente, id_usuario, id_apertura, id_cotizacion, estado
	DETALLE_VENTA	Productos de cada venta.	id_detalle_venta, id_venta, id_producto, cantidad, precio_venta, subtotal
	METODO_PAGO	Catálogo: efectivo, tarjeta, transferencia.	id_metodo, nombre, afecta_caja, requiere_referencia
	PAGO_VENTA	Pagos con que se cubre cada venta.	id_pago, id_venta, id_metodo, monto, referencia, monto_recibido
Cotizaciones	COTIZACION	Propuesta de venta con vencimiento.	id_cotizacion, id_cliente, id_usuario, fecha, fecha_vencimiento, total, estado
	DETALLE_COTIZACION	Productos de cada cotización.	id_detalle_cotizacion, id_cotizacion, id_producto, cantidad, precio, subtotal

Módulo	Entidad	Descripción	Atributos principales
Caja	CAJA	Catálogo de cajas físicas.	id_caja, nombre, estado
	APERTURA_CAJA	Turno de un usuario en una caja (apertura y cierre).	id_apertura, id_caja, id_usuario, monto_inicial, monto_esperado, monto_contado, diferencia, estado
	VALE_DESCUENTO	Vale generado por un faltante de caja.	id_vale, id_apertura, monto, estado, fecha_emision
	POLITICA_SANCION	Catálogo: sanción según número de incidencias.	id_politica, minimo_incidencias, sancion

El detalle completo de columnas, tipos y restricciones de cada tabla está en el **Anexo A. Diccionario de datos**.

2.2 Relaciones y cardinalidades

Las relaciones se implementan con **26 llaves foráneas**. La notación usada es 1:N (uno a muchos) y 1:0..1 (uno a cero o uno).

Entidad padre	Entidad hija	Cardinalidad	Significado	Llave foránea
USUARIO	USUARIO_ROL	1:N	Un usuario tiene uno o varios roles	id_usuario
ROL	USUARIO_ROL	1:N	Un rol lo tienen varios usuarios	id_rol
CLIENTES	VENTAS	1:N	Un cliente realiza varias ventas	id_cliente
CLIENTES	COTIZACION	1:N	Un cliente solicita varias cotizaciones	id_cliente
PROVEEDORES	COMPRAS	1:N	A un proveedor se le hacen varias compras	id_proveedor
USUARIO	COMPRAS / VENTAS / COTIZACION	1:N	Un usuario registra varios documentos	id_usuario

Entidad padre	Entidad hija	Cardinalidad	Significado	Llave foránea
USUARIO	VENTAS (anulación)	1:N	Un usuario solicita y otro autoriza anulaciones	id_usuario_anula, id_usuario_autoriza
USUARIO	APERTURA_CAJA / KARDEX	1:N	Un usuario abre cajas y genera movimientos	id_usuario
CAJA	APERTURA_CAJA	1:N	Una caja tiene muchas aperturas en el tiempo	id_caja
APERTURA_CAJA	VENTAS	1:N	En una apertura se hacen varias ventas	id_apertura
APERTURA_CAJA	VALE_DESCUENTO	1:0..1	Una apertura con faltante genera un solo vale	id_apertura (UNIQUE)
COTIZACION	VENTAS	1:0..1	Una cotización se factura una sola vez	id_cotizacion (UNIQUE)
COMPRAS	DETALLE_COMPRA	1:N	Una compra tiene varios productos	id_compra
VENTAS	DETALLE_VENTA	1:N	Una venta tiene varios productos	id_venta
VENTAS	PAGO_VENTA	1:N	Una venta se paga con uno o varios pagos	id_venta
COTIZACION	DETALLE_COTIZACION	1:N	Una cotización tiene varios productos	id_cotizacion
METODO_PAGO	PAGO_VENTA	1:N	Un método se usa en muchos pagos	id_metodo
PRODUCTOS	DETALLE_COMPRA / VENTA / COTIZACION	1:N	Un producto aparece en muchos documentos	id_producto
PRODUCTOS	KARDEX / AUDITORIA_PRODUCTO	1:N	Un producto tiene muchos movimientos y cambios de precio	id_producto

POLITICA_SANCION no tiene llaves foráneas: es un catálogo de parámetros que consulta la vista VW_SANCIONES_MES.

2.3 Resolución de relaciones de muchos a muchos

Una relación de muchos a muchos (M:N) no puede implementarse directamente en un modelo relacional. Cada una se resolvió con una **entidad asociativa** cuya llave combina las llaves de las dos entidades y que, cuando hace falta, guarda los datos propios de la relación:

Relación M:N	Entidad asociativa	Datos propios de la relación
Un usuario tiene varios roles y un rol lo tienen varios usuarios	USUARIO_ROL (PK compuesta: id_usuario, id_rol)	Ninguno
Una compra incluye varios productos y un producto está en varias compras	DETALLE_COMPRA	cantidad, precio_costo, subtotal
Una venta incluye varios productos y un producto está en varias ventas	DETALLE_VENTA	cantidad, precio_venta, subtotal
Una cotización incluye varios productos y un producto está en varias cotizaciones	DETALLE_COTIZACION	cantidad, precio, subtotal
Una venta se paga con varios métodos y un método se usa en varias ventas	PAGO_VENTA	monto, referencia, monto_recibido

2.4 Diagrama Entidad–Relación

El modelo conceptual se representa con la **notación de Chen**: cada **rectángulo** es una entidad, cada **rombo** es una relación y cada **óvalo** es un atributo. La llave primaria aparece **subrayada** y los datos calculados (stock, totales, subtotales, monto esperado y diferencia) van en **óvalo punteado**, porque se derivan de otros datos (ver sección 3.5). Junto a cada línea se indica la cardinalidad (1, N o M).

Las llaves foráneas **no** aparecen como atributos: en el modelo conceptual las representa cada rombo. Por ejemplo, VENTAS no muestra id_cliente; esa conexión

es el rombo REALIZA entre CLIENTES y VENTAS. Las cuatro relaciones de muchos a muchos (USUARIO_ROL, DETALLE_COMPRA, DETALLE_VENTA y DETALLE_COTIZACION) llevan sus propios atributos y en el modelo relacional se convierten en tablas asociativas: **16 entidades + 4 relaciones M:N = 20 tablas**, unidas por las **26 llaves foráneas** de la base de datos. PAGO_VENTA también resuelve una relación M:N (venta–método de pago), pero se dibuja como entidad porque tiene identificador propio (id_pago) y una misma venta puede tener dos pagos con el mismo método (por ejemplo, dos tarjetas).

La Figura 1 muestra el diagrama completo (entidades, relaciones y cardinalidades). Para que los atributos se lean con claridad, las Figuras 2 a 5 presentan el mismo modelo por módulo; cada entidad muestra sus atributos en un solo diagrama y en los demás aparece como un rectángulo vacío que indica con qué se relaciona. POLITICA_SANCION no tiene relaciones porque es un catálogo que solo consulta la vista VW_SANCIONES_MES.

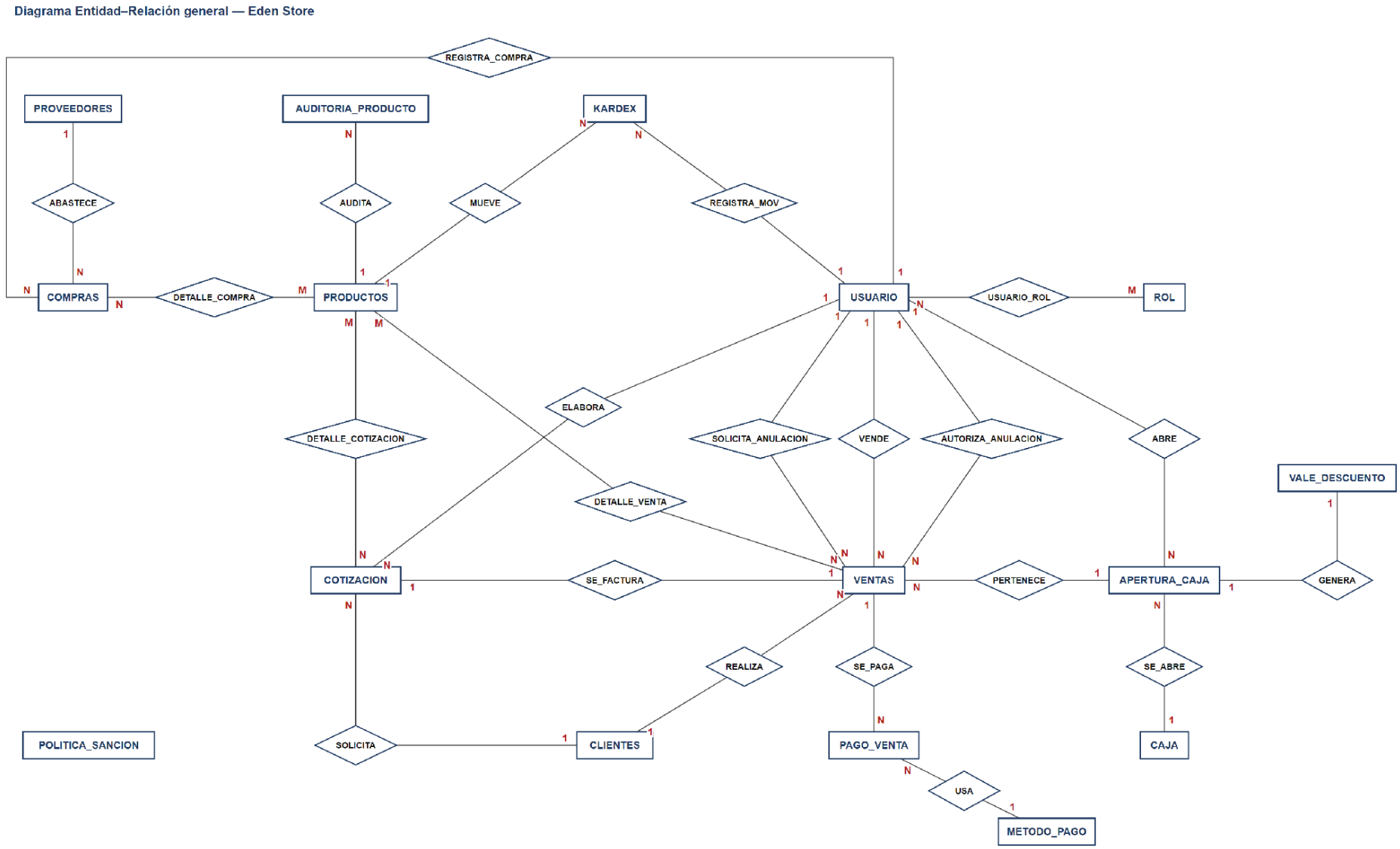


Figura 1. Diagrama Entidad–Relación general (notación de Chen).

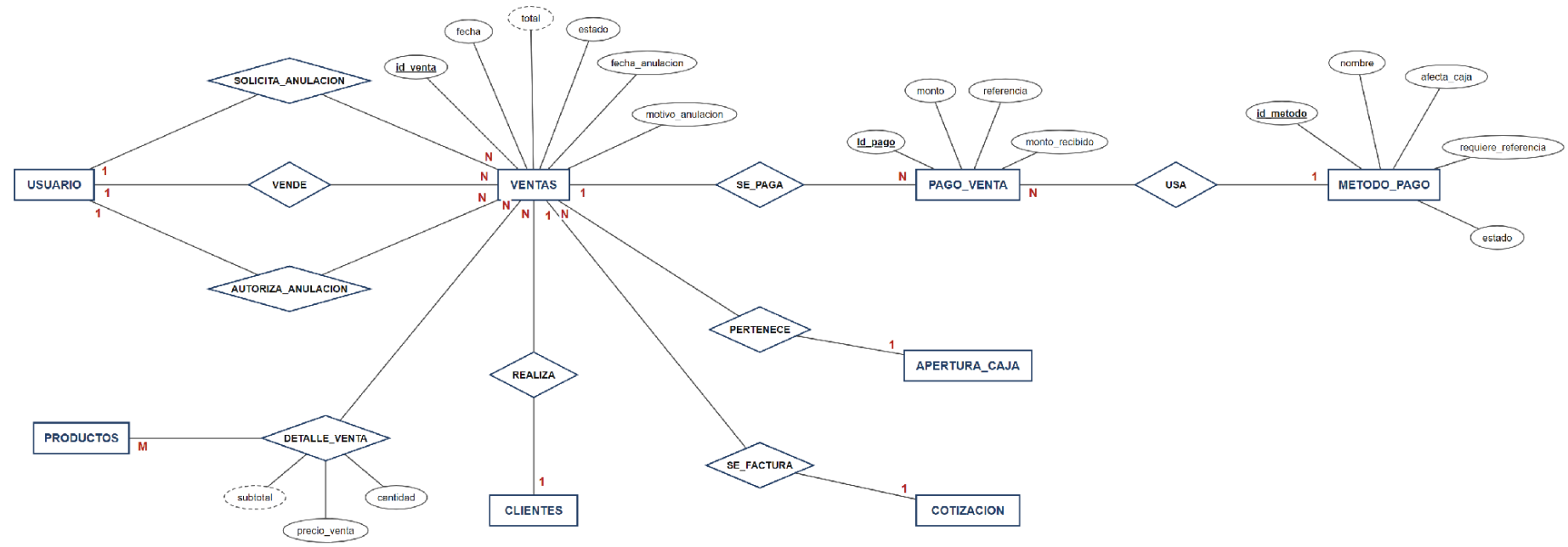


Figura 2. Diagrama ER — módulo de Ventas.

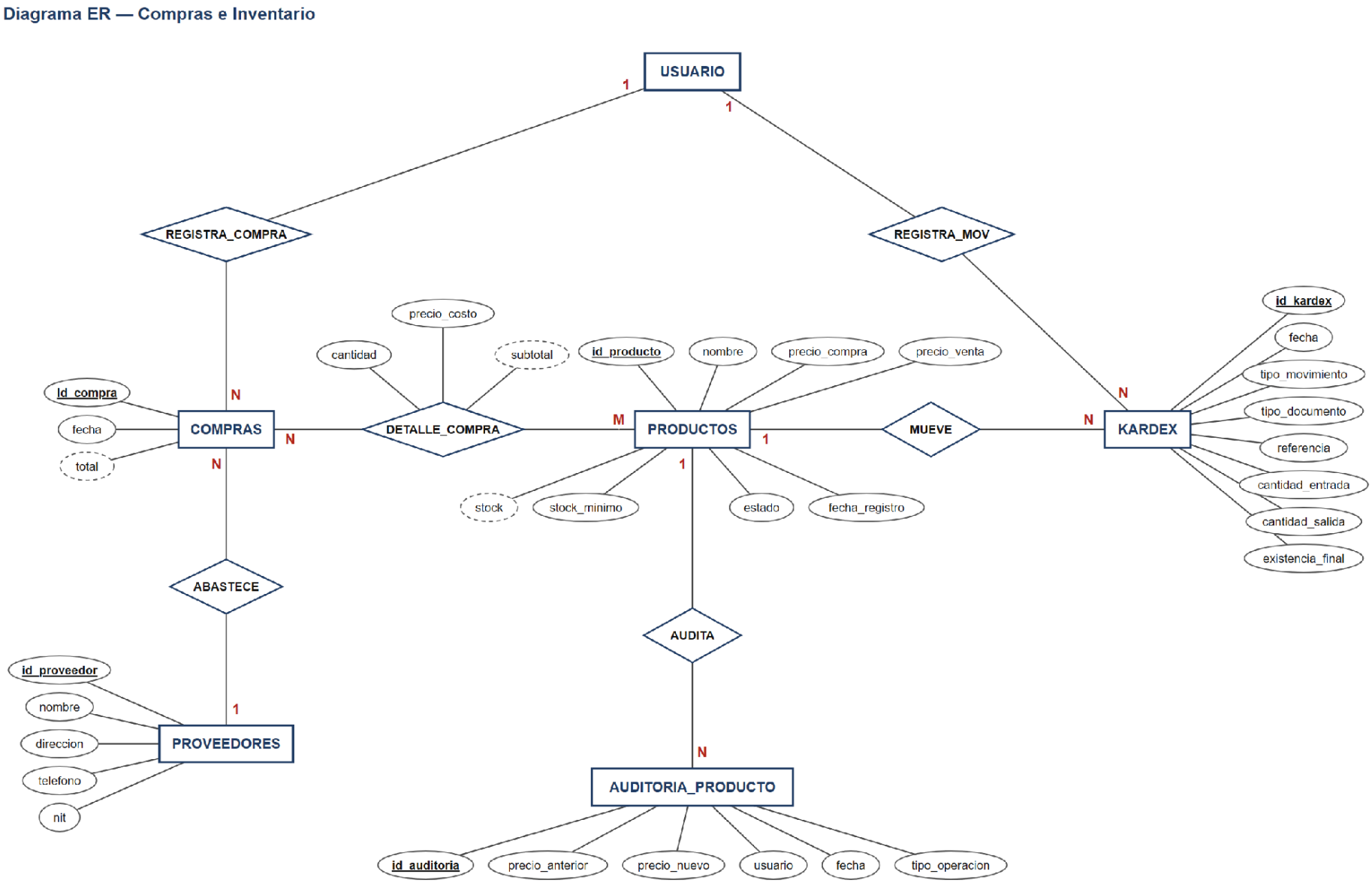


Figura 3. Diagrama ER — módulos de Compras e Inventario.

Diagrama ER — Cotizaciones y Clientes

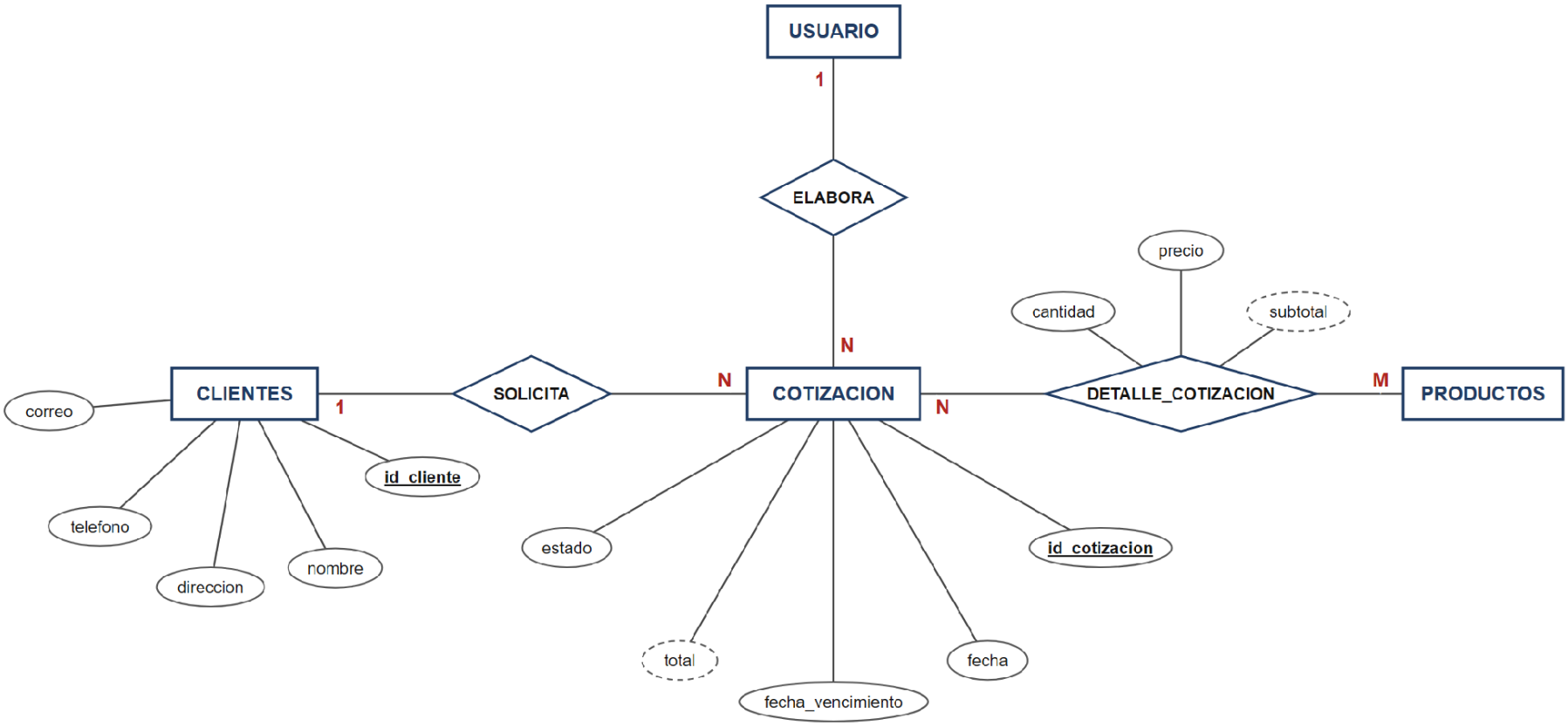


Figura 4. Diagrama ER — módulo de Cotizaciones y Clientes.

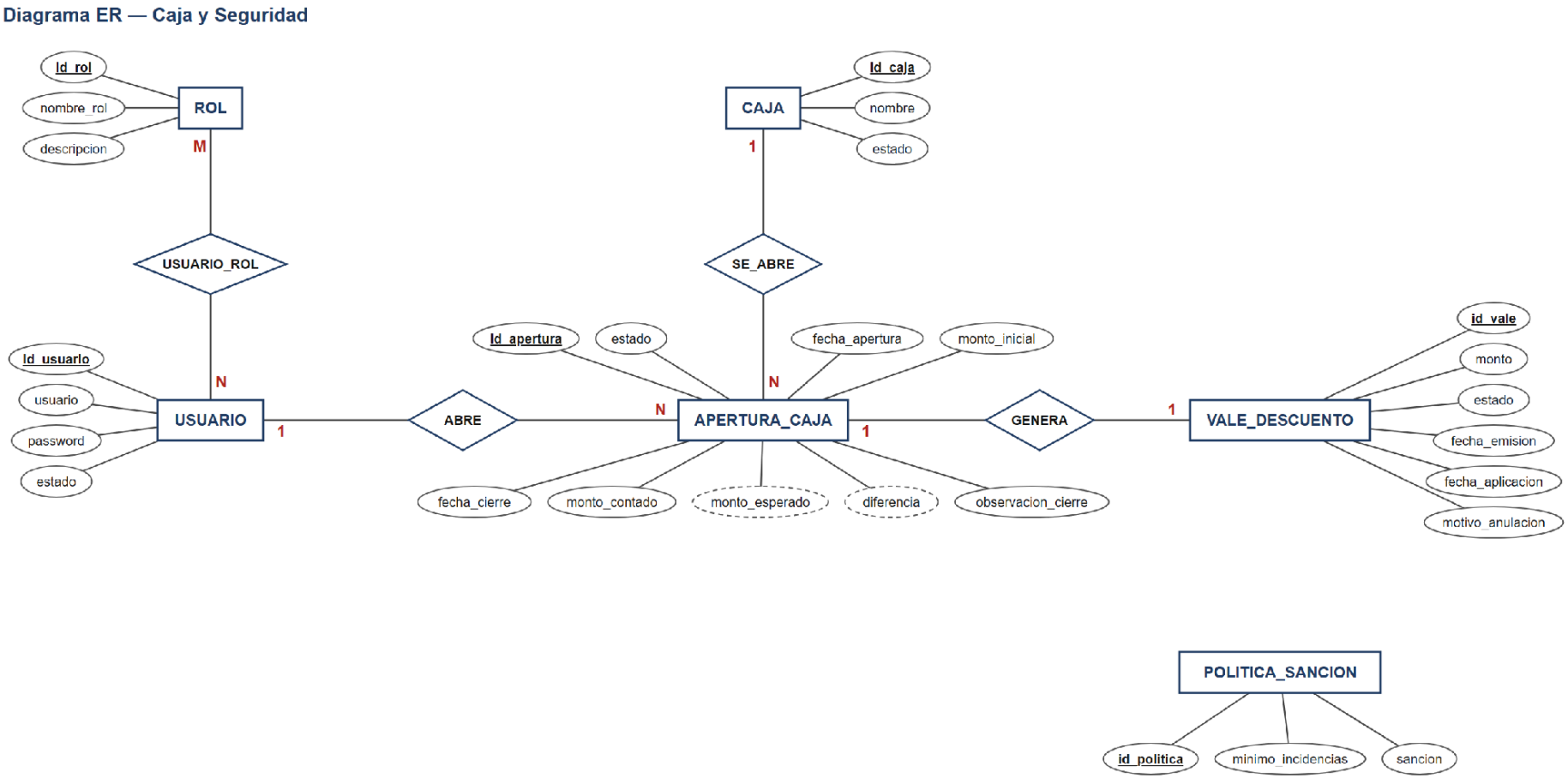


Figura 5. Diagrama ER — módulos de Caja y Seguridad.

2.5 Modelo relacional

Cada entidad se transformó en una tabla. Las llaves primarias se indican **subrayadas en el diagrama** y aquí en negrita; las llaves foráneas se marcan con (FK → tabla).

- ROL(**id_rol**, nombre_rol, descripcion)
- USUARIO(**id_usuario**, usuario, password, estado)
- USUARIO_ROL(**id_usuario** (FK → USUARIO), **id_rol** (FK → ROL))
- CLIENTES(**id_cliente**, nombre, direccion, telefono, correo)
- PROVEEDORES(**id_proveedor**, nombre, direccion, telefono, nit)
- PRODUCTOS(**id_producto**, nombre, precio_compra, precio_venta, stock, estado, fecha_registro, stock_minimo)
- CAJA(**id_caja**, nombre, estado)
- APERTURA_CAJA(**id_apertura**, id_caja (FK → CAJA), id_usuario (FK → USUARIO), fecha_apertura, monto_inicial, fecha_cierre, monto_esperado, monto_contado, diferencia, estado, observacion_cierre)
- COTIZACION(**id_cotizacion**, id_cliente (FK → CLIENTES), id_usuario (FK → USUARIO), fecha, fecha_vencimiento, total, estado)
- DETALLE_COTIZACION(**id_detalle_cotizacion**, id_cotizacion (FK → COTIZACION), id_producto (FK → PRODUCTOS), cantidad, precio, subtotal)
- COMPRAS(**id_compra**, fecha, total, id_proveedor (FK → PROVEEDORES), id_usuario (FK → USUARIO))
- DETALLE_COMPRA(**id_detalle_compra**, id_compra (FK → COMPRAS), id_producto (FK → PRODUCTOS), cantidad, precio_costo, subtotal)
- VENTAS(**id_venta**, fecha, total, id_cliente (FK → CLIENTES), id_usuario (FK → USUARIO), id_apertura (FK → APERTURA_CAJA), id_cotizacion (FK → COTIZACION), estado, fecha_anulacion, id_usuario_anula (FK → USUARIO), id_usuario_autoriza (FK → USUARIO), motivo_anulacion)
- DETALLE_VENTA(**id_detalle_venta**, id_venta (FK → VENTAS), id_producto (FK → PRODUCTOS), cantidad, precio_venta, subtotal)

- METODO_PAGO(**id_metodo**, nombre, afecta_caja, requiere_referencia, estado)
- PAGO_VENTA(**id_pago**, id_venta (FK → VENTAS), id_metodo (FK → METODO_PAGO), monto, referencia, monto_recibido)
- KARDEX(**id_kardex**, id_producto (FK → PRODUCTOS), fecha, tipo_movimiento, tipo_documento, referencia, cantidad_entrada, cantidad_salida, existencia_final, id_usuario (FK → USUARIO))
- AUDITORIA_PRODUCTO(**id_auditoria**, id_producto (FK → PRODUCTOS), precio_anterior, precio_nuevo, usuario, fecha, tipo_operacion)
- VALE_DESCUENTO(**id_vale**, id_apertura (FK → APERTURA_CAJA), monto, estado, fecha_emision, fecha_aplicacion, motivo_anulacion)
- POLITICA_SANCION(**id_politica**, minimo_incidencias, sancion)

La Figura 6 es el modelo relacional generado en **Oracle SQL Developer Data Modeler** por ingeniería inversa desde el esquema real SISTEMA_VENTAS, por lo que coincide exactamente con la base de datos implementada. En cada tabla, **P** indica llave primaria, **F** llave foránea, **U** restricción UNIQUE y el asterisco (*) un campo obligatorio (NOT NULL). Las líneas unen cada llave foránea con la tabla a la que apunta; el extremo con «pata de gallo» es el lado *muchos* de la relación.

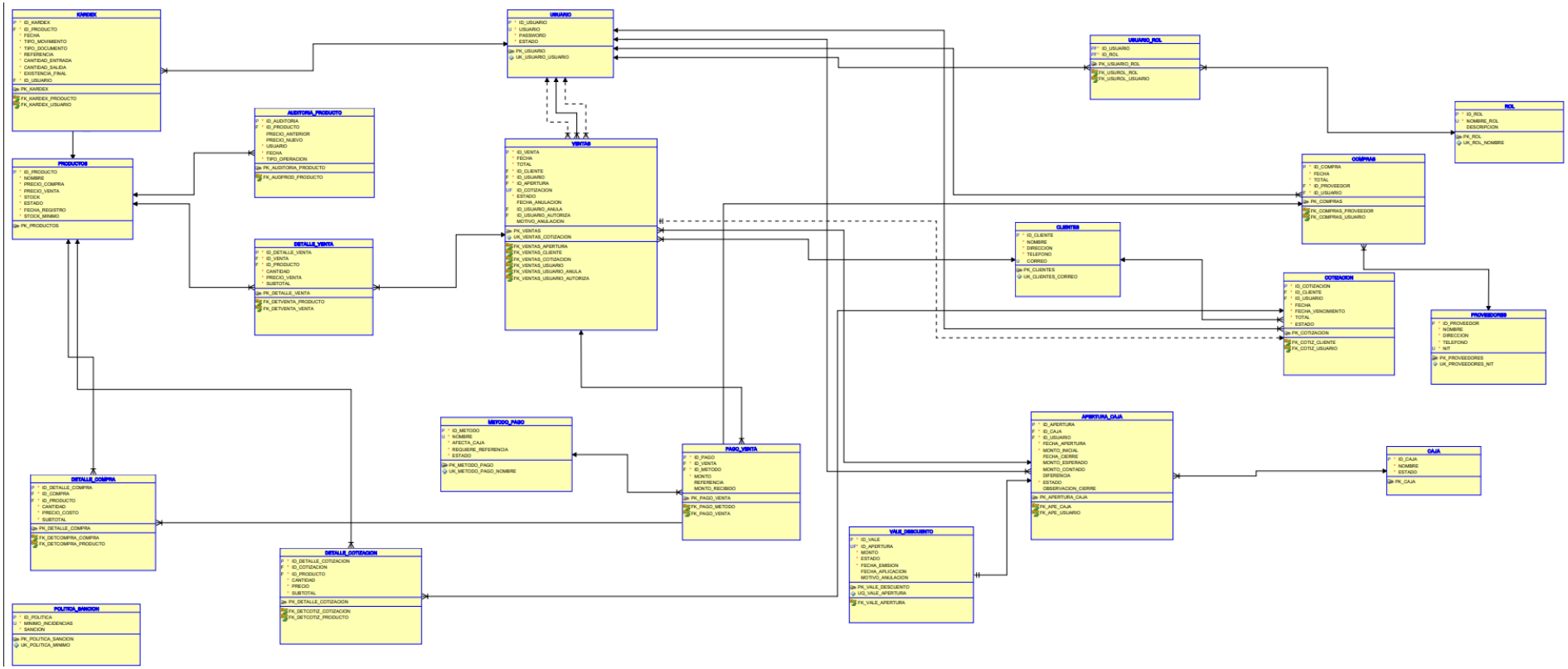


Figura 6. Modelo relacional (Oracle SQL Developer Data Modeler).

2.6 Modelo físico

El modelo físico define los tipos de datos de Oracle y todas las restricciones. Se usaron NUMBER(10) para identificadores, NUMBER(12,2) para montos en quetzales, NUMBER(8) para cantidades, VARCHAR2 para textos, CHAR(1) para estados codificados y DATE para fechas.

Tipo de restricción	Cantidad	Ejemplos
PRIMARY KEY	20	Una por tabla; USUARIO_ROL usa llave compuesta (id_usuario, id_rol).
FOREIGN KEY	26	fk_ventas_apertura, fk_detventa_producto, fk_pago_metodo.
UNIQUE	8	uk_usuario_usuario, uk_proveedores_nit, uk_ventas_cotizacion, uq_vale_apertura.
CHECK	35	`chk_productos_stock (stock ≥ 0)`, `chk_productos_precios`, `chk_kardex_coherencia`.
CHECK de varias columnas	8 (incluidos arriba)	chk_ventas_anulacion, chk_vale_aplicado, chk_vale_anulado, chk_ape_cierre.
NOT NULL	82	Todos los datos obligatorios (nombre, total, cantidad, fecha…).
DEFAULT	—	estado DEFAULT 'A', fecha DEFAULT SYSDATE, stock DEFAULT 0.

Ejemplo de una restricción CHECK que involucra varias columnas: una venta activa no puede tener datos de anulación y una anulada debe tenerlos todos.

```
CONSTRAINT chk_ventas_anulacion CHECK (
   (estado = 'A' AND fecha_anulacion IS NULL AND id_usuario_anula IS
NULL
            AND id_usuario_autoriza IS NULL AND motivo_anulacion IS
NULL)
   OR
   (estado = 'N' AND fecha_anulacion IS NOT NULL AND id_usuario_anula IS
NOT NULL
            AND id_usuario_autoriza IS NOT NULL AND motivo_anulacion IS
NOT NULL))
```

La Figura 7 muestra el modelo físico: además de las llaves, incluye el **tipo de dato de Oracle** de cada columna y, al pie de cada tabla, sus restricciones (PK, UK, FK) y los **índices** creados (IDX_...). El DDL que genera Data Modeler a partir de este modelo se guardó en database/diagramas/modelo_fisico_ddl.sql.

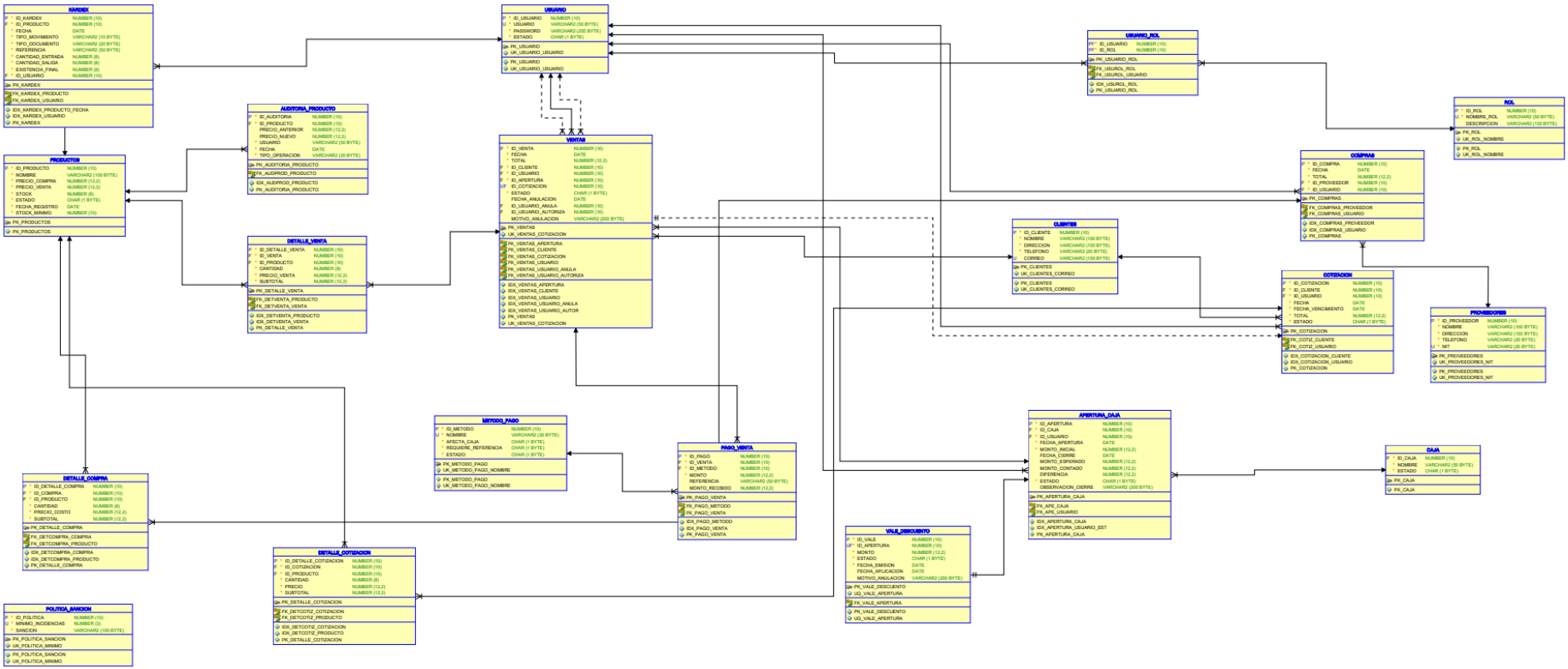


Figura 7. Modelo físico con tipos de datos, restricciones e índices (Data Modeler).

3. Normalización

Para demostrar la normalización se parte de cómo Eden Store registraba una venta en su hoja de cálculo: una sola fila por venta con todos los datos juntos. Desde esa relación sin normalizar se aplican las tres primeras formas normales hasta llegar a las tablas implementadas.

3.1 Relación sin normalizar (0FN)

La planilla de ventas tenía estas columnas:

```
PLANILLA_VENTA ( no_venta, fecha, nombre_cliente, direccion_cliente,
telefono_cliente,
        vendedor, roles_vendedor, caja,
        productos { codigo, nombre_producto, cantidad, precio, subtotal },
        pagos { metodo, monto, referencia, recibido },
        total, cambio, faltante_caja )
```

Ejemplo de una fila real:

no_venta	cliente	vendedor / roles	productos	pagos	total
10	Juan Pérez, Calle 5, Zona 11, 77885214	cajero1 / CAJERO	10 Memoria USB 64GB a Q55 = Q550; 2 Teclado Mecánico a Q200 = Q400	Efectivo Q475 (recibió Q500, cambio Q25); Tarjeta AUT-000010 Q475	950

Problemas: los productos y los pagos son **grupos repetitivos** (una venta tiene varios), roles_vendedor guarda varios valores en una celda, y los datos del cliente y del producto se repiten en cada venta, por lo que cambiar un teléfono o un nombre obliga a modificar muchas filas (anomalía de actualización).

3.2 Primera forma normal (1FN)

Regla: todos los atributos deben ser atómicos y no puede haber grupos repetitivos. Cada fila debe identificarse con una llave primaria.

- Los productos se separan en una fila por producto vendido: la llave pasa a ser **(no_venta, codigo_producto)**.

- Los pagos se separan en una fila por pago: llave **(no_venta, metodo)**.
- Los roles se separan en una fila por rol: llave **(usuario, rol)**.

```
VENTA_PRODUCTO ( no_venta, codigo_producto, fecha, nombre_cliente,
direccion_cliente,
                telefono_cliente, vendedor, caja, nombre_producto, precio_catalogo,
                cantidad, precio, subtotal, total )
VENTA_PAGO     ( no_venta, metodo, monto, referencia, recibido )
USUARIO_ROL    ( usuario, rol )
```

La tabla ya está en 1FN, pero todavía repite la fecha, el cliente y el total en cada producto de la misma venta.

3.3 Segunda forma normal (2FN)

Regla: estar en 1FN y que cada atributo que no es llave dependa de **toda** la llave primaria, no solo de una parte (sin dependencias parciales).

En VENTA_PRODUCTO la llave es (no_venta, codigo_producto). Se identifican estas dependencias funcionales:

Dependencia funcional	Tipo	Acción
no_venta → fecha, nombre_cliente, direccion_cliente, telefono_cliente, vendedor, caja, total	Parcial	Se separa en VENTAS
codigo_producto → nombre_producto, precio_catalogo	Parcial	Se separa en PRODUCTOS
(no_venta, codigo_producto) → cantidad, precio, subtotal	Completa	Queda en DETALLE_VENTA
metodo → requiere_referencia, afecta_caja	Parcial (en VENTA_PAGO)	Se separa en METODO_PAGO

```
VENTAS        ( no_venta, fecha, nombre_cliente, direccion_cliente,
telefono_cliente,
                vendedor, caja, total )
DETALLE_VENTA ( no_venta, codigo_producto, cantidad, precio_venta,
subtotal )
PRODUCTOS     ( codigo_producto, nombre, precio_venta, ... )
PAGO_VENTA    ( id_pago, no_venta, id_metodo, monto, referencia,
```

monto_recibido)
METODO_PAGO (id_metodo, nombre, afecta_caja, requiere_referencia)

Se aplicó el mismo análisis a compras (COMPRAS / DETALLE_COMPRA) y cotizaciones (COTIZACION / DETALLE_COTIZACION). En la implementación, los detalles usan una llave sustituta (id_detalle_venta), pero conceptualmente cada fila de detalle sigue identificada por la venta y el producto.

Nota sobre el precio en el detalle: DETALLE_VENTA.precio_venta no es una dependencia parcial. Guarda el precio **en el momento de la venta**, que depende de la combinación venta–producto; si el precio del catálogo cambia mañana, la venta de hoy debe conservar su precio original.

3.4 Tercera forma normal (3FN)

Regla: estar en 2FN y que ningún atributo que no es llave dependa de otro atributo que tampoco es llave (sin dependencias transitivas).

Dependencia transitiva encontrada	Solución (tabla resultante)
no_venta → id_cliente → nombre, dirección, teléfono, correo	CLIENTES; VENTAS conserva solo id_cliente
no_venta → id_usuario → usuario, contraseña, estado	USUARIO; VENTAS conserva solo id_usuario
no_venta → id_apertura → id_caja → nombre de la caja	APERTURA_CAJA y CAJA; VENTAS conserva id_apertura
id_compra → id_proveedor → nombre, NIT, dirección	PROVEEDORES; COMPRAS conserva id_proveedor
id_pago → id_metodo → afecta_caja, requiere_referencia	METODO_PAGO; PAGO_VENTA conserva id_metodo
usuario → rol → descripción del rol	ROL; USUARIO_ROL guarda solo las llaves
incidencias del mes → sanción sugerida	POLITICA_SANCION (catálogo de parámetros)

Con estas separaciones se obtienen las 20 tablas del modelo. En todas ellas cada atributo depende **de la llave, de toda la llave y nada más que de la llave**.

3.5 Decisiones de diseño sobre datos calculados

Durante la normalización se evaluó cada dato que puede calcularse a partir de otros:

Dato	Decisión	Justificación
Cambio entregado al cliente	**No se guarda**	`Se calcula como monto_recibido − monto al mostrar el recibo.`
Faltantes y sobrantes de caja	**No se guardan** en una tabla	Se calculan en la vista VW_INCIDENCIAS_CAJA a partir de la diferencia de cada cierre.
Sanción sugerida	**No se guarda**	La vista VW_SANCIONES_MES la deduce de las incidencias y de POLITICA_SANCION.
Stock del producto	Se guarda (redundancia controlada)	Consultarlo sumando todo el Kardex en cada venta sería lento; solo lo modifican los procedimientos y se verifica con la consulta 11.2.
Total de la venta, compra y cotización	Se guarda (redundancia controlada)	Es el valor oficial del documento; lo acumulan los procedimientos en la misma transacción. Consulta 11.3.
Monto esperado y diferencia de caja	Se guardan al cerrar	Son una fotografía del cierre (lo que se contó ese día); los calcula SP_CERRAR_CAJA. Consulta 11.5.
Existencia final en el Kardex	Se guarda	Es histórica: muestra el stock que quedó después de cada movimiento.

Las redundancias controladas no rompen la 3FN en la práctica porque **ningún usuario puede modificarlas directamente**: solo cambian dentro de los procedimientos almacenados, en la misma transacción que el dato de origen, y las consultas de verificación 11.2 a 11.5 del script comprueban que siempre coinciden (todas devuelven OK).

4. Implementación en Oracle

4.1 Entorno

Elemento	Valor
Motor	Oracle Database 21c Express Edition

Elemento	Valor
Base de datos conectable (PDB)	XEPDB1
Esquema dueño de los objetos	SISTEMA_VENTAS
Herramientas	SQL Developer, SQL Developer Data Modeler, SQL*Plus, Data Pump
Script de instalación	database/proyecto_sistema_ventas.sql (se ejecuta completo con F5)

4.2 Organización del script

El script se puede ejecutar de nuevo sobre un esquema ya usado: la sección 1 elimina los objetos anteriores ignorando los errores de «el objeto no existe».

Sección	Contenido
1	Limpieza de objetos anteriores
2	Secuencias (16)
3	Tablas y restricciones (20 tablas)
4	Índices (23 sobre llaves foráneas y búsquedas frecuentes)
5	Vistas (6)
6	Funciones (2)
7	Procedimientos almacenados (15)
8	Triggers (2)
9	Job programado con DBMS_SCHEDULER
10	Datos de prueba (agosto a octubre de 2026)
11	Consultas de verificación y de explotación de la información
12	Pruebas de reglas de negocio y manejo de errores
Anexo A	Seguridad: roles, usuarios y privilegios (se ejecuta como SYSDBA)
Anexo B	Respaldo y restauración con Data Pump

4.3 Secuencias e índices

Cada tabla con identificador generado tiene su secuencia (SEQ_VENTA, SEQ_KARDEX, etc.). Además de los índices que Oracle crea para las llaves primarias y únicas, se crearon **23 índices** sobre todas las llaves foráneas, incluidos dos compuestos para las consultas más frecuentes:

- KARDEX(id_producto, fecha): historial de un producto ordenado por fecha.

- APERTURA_CAJA(id_usuario, estado): buscar la caja abierta de un usuario antes de cada venta.

La consulta 11.14 del script comprueba que no queda ninguna llave foránea sin índice (devuelve 0 filas).

4.4 Vistas

Vista	Propósito	Uso en la aplicación
VW_RESUMEN_VENTAS	Ventas con cliente, usuario, total y estado (JOIN de 3 tablas).	Lista de ventas
VW_PRODUCTOS_BAJO_STOCK	Productos con stock menor o igual al mínimo.	Filtro «Bajo stock» e Inicio
VW_VENTAS_POR_PRODUCTO	Unidades e ingresos por producto (solo ventas activas).	Reportes
VW_INCIDENCIAS_CAJA	Faltantes y sobrantes calculados desde los cierres.	Caja → Incidencias
VW_ANULACIONES_VENTA	Ventas anuladas: quién vendió, quién solicitó y quién autorizó.	Caja → Incidencias
VW_SANCIONES_MES	Incidencias por usuario y mes con la sanción sugerida.	Caja → Incidencias

4.5 Funciones

Función	Devuelve	Dónde se usa
FN_STOCK_PRODUCTO(id_producto)	Existencia actual; lanza ORA-20001 si el producto no existe.	Lista de productos, prueba 12.3
FN_EFECTIVO_APERTURA(id_apertura)	Efectivo de las ventas activas de una apertura.	Cierre de caja, resumen de caja, consulta 11.5

```
CREATE OR REPLACE FUNCTION FN_STOCK_PRODUCTO (
   p_id_producto IN PRODUCTOS.id_producto%TYPE
) RETURN NUMBER
IS
   v_stock PRODUCTOS.stock%TYPE;
BEGIN
   SELECT stock INTO v_stock FROM PRODUCTOS WHERE id_producto =
p_id_producto;
```

```
    RETURN v_stock;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'No existe el producto
solicitado.');
END FN_STOCK_PRODUCTO;
```

4.6 Procedimientos almacenados

Los procedimientos que forman parte de una operación mayor **no hacen COMMIT**: la aplicación llama a varios de ellos y confirma o deshace todo junto. Los procedimientos independientes sí confirman su propio trabajo.

Procedimiento	COMMIT propio	Propósito
SP_REGISTRAR_CLIENTE	Sí	Registro de clientes.
SP_REGISTRAR_DETALLE_COMPRA	Sí	Compra de un producto (laboratorio 3).
SP_REGISTRAR_DETALLE_VENTA	Sí	Venta de un producto (laboratorio 3).
SP_VENTA_TRANSACCION	Sí	Demostración de COMMIT y ROLLBACK (laboratorio 3).
SP_PROCESAR_VENTA	Sí	Proceso integrador de un producto (laboratorio 3).
SP_AGREGAR_DETALLE_COMPRA	No	Detalle de compra + aumento de stock + ENTRADA en el Kardex.
SP_CREAR_VENTA	No	Encabezado: valida cliente, usuario activo, caja abierta y cotización (OUT id_venta).
SP_AGREGAR_DETALLE_VENTA	No	Detalle con precio del catálogo + disminución de stock + SALIDA en el Kardex.
SP_AGREGAR_PAGO_VENTA	No	Pago en efectivo, tarjeta o transferencia, con sus validaciones.
SP_FINALIZAR_VENTA	No	Verifica que los pagos sumen el total y actualiza el monto esperado de la caja.

Procedimiento	COMMIT propio	Propósito
SP_ANULAR_VENTA	Sí	Anulación autorizada por un administrador; un cursor devuelve el stock de cada producto.
SP_CADUCAR_COTIZACIONES	Sí	Marca como caducadas las cotizaciones vencidas.
SP_ABRIR_CAJA	Sí	Apertura de caja (OUT id_apertura).
SP_CERRAR_CAJA	Sí	Cierre: calcula el monto esperado y la diferencia.
SP_APLICAR_VALES_MES	Sí	Proceso de fin de mes: aplica los vales pendientes (OUT cantidad).

Fragmento de SP_AGREGAR_DETALLE_VENTA, el procedimiento que actualiza existencias y Kardex en cada venta:

```
IF v_stock < p_cantidad THEN
   RAISE_APPLICATION_ERROR(-20004, 'Stock insuficiente de ' || v_nombre
||
      '. Disponible: ' || v_stock || ', solicitado: ' || p_cantidad || '.');
END IF;

v_subtotal := p_cantidad * v_precio;

-- 1. Detalle (el precio sale del catálogo, no del usuario)
INSERT INTO DETALLE_VENTA (id_detalle_venta, id_venta, id_producto,
cantidad, precio_venta, subtotal)
VALUES (SEQ_DETALLE_VENTA.NEXTVAL, p_id_venta, p_id_producto,
p_cantidad, v_precio, v_subtotal);

-- 2. Disminuir el stock
UPDATE PRODUCTOS SET stock = stock - p_cantidad WHERE id_producto
= p_id_producto;

-- 3. Total de la venta
UPDATE VENTAS SET total = total + v_subtotal WHERE id_venta =
p_id_venta;

-- 4. SALIDA en el Kardex
```

```
INSERT INTO KARDEX (id_kardex, id_producto, fecha, tipo_movimiento,
tipo_documento,
                referencia, cantidad_entrada, cantidad_salida, existencia_final,
id_usuario)
VALUES (SEQ_KARDEX.NEXTVAL, p_id_producto, SYSDATE, 'SALIDA',
'VENTA',
       'VENTA #' || p_id_venta, 0, p_cantidad, v_stock - p_cantidad,
v_id_usuario);
```

El producto se lee con SELECT ... FOR UPDATE, que bloquea la fila hasta terminar la transacción. Así, si dos cajeros venden el último producto al mismo tiempo, el segundo espera y luego recibe el error de stock insuficiente.

4.7 Triggers

Trigger	Evento	Acción
TRG_AUDITORIA_PRECIO	AFTER UPDATE OF precio_venta ON PRODUCTOS	Inserta en AUDITORIA_PRODUCTO el precio anterior, el nuevo y el usuario de la aplicación (CLIENT_IDENTIFIER).
TRG_VALE_FALTANTE	AFTER UPDATE OF estado ON APERTURA_CAJA	Al cerrar una caja con diferencia negativa, genera el vale de descuento por el faltante.

```
CREATE OR REPLACE TRIGGER TRG_VALE_FALTANTE
AFTER UPDATE OF estado ON APERTURA_CAJA
FOR EACH ROW
WHEN (OLD.estado = 'A' AND NEW.estado = 'C' AND NEW.diferencia < 0)
BEGIN
   INSERT INTO VALE_DESCUENTO (id_vale, id_apertura, monto, estado,
fecha_emision)
   VALUES (SEQ_VALE.NEXTVAL, :NEW.id_apertura,
ABS(:NEW.diferencia), 'P', SYSDATE);
END TRG_VALE_FALTANTE;
```

4.8 Tarea programada

JOB_CADUCAR_COTIZACIONES se creó con DBMS_SCHEDULER y ejecuta SP_CADUCAR_COTIZACIONES todos los días a las 00:05, aunque nadie esté usando la aplicación.

4.9 Transacciones: COMMIT y ROLLBACK

Una venta involucra varias tablas (VENTAS, DETALLE_VENTA, PRODUCTOS, KARDEX, PAGO_VENTA y APERTURA_CAJA). Para que nunca quede registrada a medias, la aplicación ejecuta todos los procedimientos en una sola transacción y decide al final:

```
try:
    cursor.callproc("SP_CREAR_VENTA", [id_cliente, id_usuario,
id_cotizacion, id_venta_var])
    for linea in lineas:
        cursor.callproc("SP_AGREGAR_DETALLE_VENTA", [id_venta,
linea["id_producto"], linea["cantidad"]])
    for pago in pagos:
        cursor.callproc("SP_AGREGAR_PAGO_VENTA", [id_venta,
pago["id_metodo"], pago["monto"], ...])
    cursor.callproc("SP_FINALIZAR_VENTA", [id_venta])
    conexion.commit()       # todo salió bien: se confirma la venta completa
except oracledb.DatabaseError:
    conexion.rollback()     # cualquier error: se deshace todo
    raise
```

Si el tercer producto no tiene stock, se deshacen también el encabezado, los dos primeros productos, sus movimientos de Kardex y la disminución de existencias. La prueba 12.1 del script lo demuestra (sección 8.3).

4.10 Manejo de excepciones

Los bloques PL/SQL usan excepciones predefinidas de Oracle y errores propios:

- NO_DATA_FOUND: cuando no existe el producto, la venta o el usuario buscado; se convierte en un mensaje claro.
- DUP_VAL_ON_INDEX: cuando se intenta repetir un valor único (prueba 12.13).

- RAISE_APPLICATION_ERROR con códigos propios entre -20001 y -20099 para cada regla de negocio, por ejemplo -20004 (stock insuficiente), -20013 (sin caja abierta), -20056 (pagos distintos al total), -20062 (caja ya cerrada) y -20063 (anulación sin autorización).

En la aplicación, el archivo models/errores.py traduce estos errores y los de restricciones (ORA-02290, ORA-02292, ORA-00001) a mensajes en español para el usuario.

5. Consultas SQL y explotación de la información

Técnica	Dónde se aplica
INSERT, UPDATE, DELETE, SELECT	Modelos de la aplicación (models/*_model.py) y procedimientos almacenados.
JOIN de varias tablas	Kardex (KARDEX + PRODUCTOS + USUARIO), lista de ventas, reportes y todas las vistas.
Funciones de agregación (SUM, COUNT, AVG, MAX, MIN)	Reporte «Resumen de ventas», valor del inventario, resumen de caja.
GROUP BY y HAVING	Reporte «Mejores clientes» (HAVING SUM(total) ≥ monto), VW_SANCIONES_MES.
Subconsultas	Reporte «Productos nunca vendidos» (NOT IN), VW_SANCIONES_MES.
Agregación condicional (CASE)	Indicadores de ventas activas y anuladas en una sola consulta.
Vistas	Las 6 vistas de la sección 4.4.

Ejemplo con **JOIN, GROUP BY y HAVING**: clientes que han comprado más de Q1,000 en ventas activas.

```
SELECT c.nombre, COUNT(v.id_venta) AS compras, SUM(v.total) AS
total_comprado
FROM CLIENTES c
JOIN VENTAS v ON c.id_cliente = v.id_cliente
WHERE v.estado = 'A'
GROUP BY c.nombre
```

```
HAVING SUM(v.total) > 1000
ORDER BY total_comprado DESC;
```

Ejemplo con **subconsulta**: productos que no aparecen en ninguna venta activa.

```
SELECT p.id_producto, p.nombre, p.stock
FROM PRODUCTOS p
WHERE p.id_producto NOT IN (SELECT d.id_producto
                            FROM DETALLE_VENTA d
                            JOIN VENTAS v ON v.id_venta = d.id_venta
                            WHERE v.estado = 'A');
```

6. Seguridad

6.1 Seguridad en Oracle

Se aplicó el **principio de menor privilegio**: cada rol de base de datos recibe solo los permisos que necesita sobre las tablas del esquema SISTEMA_VENTAS (Anexo A del script, ejecutado como SYSDBA).

Rol de Oracle	Privilegios principales	Usuario de BD
R_CONSULTOR_PRODUCTO	SELECT sobre PRODUCTOS.	usr_consulta
R_VENDEDOR	SELECT/INSERT/UPDATE en CLIENTES y COTIZACION; SELECT/INSERT en VENTAS, DETALLE_VENTA y PAGO_VENTA; SELECT en PRODUCTOS.	usr_vendedor
R_CAJERO	SELECT/INSERT/UPDATE en APERTURA_CAJA; SELECT en CAJA, VENTAS, CLIENTES y PAGO_VENTA.	—
R_BODEGUERO	SELECT/INSERT/UPDATE en PRODUCTOS, COMPRAS y DETALLE_COMPRA; SELECT/INSERT en KARDEX.	usr_bodeguero
R_ADMINISTRADOR	SELECT sobre todas las tablas de control y las vistas de reportes.	—

Evidencia: conectado como usr_vendedor, la consulta SELECT * FROM SISTEMA_VENTAS.CLIENTES funciona, pero SELECT * FROM SISTEMA_VENTAS.USUARIO devuelve **ORA-00942: la tabla o vista no existe**, porque ese rol no tiene permiso sobre la tabla de usuarios. El script también incluye un ejemplo de REVOKE.

6.2 Seguridad en la aplicación

- **Contraseñas con hash:** se guardan cifradas con scrypt (werkzeug.security); nunca en texto plano.
- **Inicio de sesión:** valida la contraseña y que el usuario esté activo; un usuario inactivo no puede entrar.
- **Control de acceso por rol:** antes de cada petición (before_request) se revisa si el rol puede usar el módulo; el menú solo muestra los módulos permitidos.
- **Anulaciones:** la aplicación pide el usuario y la contraseña de un administrador; además, SP_ANULAR_VENTA vuelve a comprobar en Oracle que quien autoriza sea un administrador activo.
- **Inyección SQL:** todas las consultas usan variables de enlace (:nombre), nunca concatenación de valores.
- **Credenciales fuera del repositorio:** config.py no se sube a GitHub; se incluye config.example.py como plantilla.
- **Auditoría:** el usuario de la aplicación se envía a Oracle (client_identifier) para que el trigger de auditoría registre quién cambió cada precio.

Módulo	Roles con acceso
Clientes	Administrador, Vendedor, Cajero
Proveedores	Administrador, Bodeguero
Productos	Administrador, Bodeguero, Vendedor (solo consulta)
Compras	Administrador, Bodeguero
Ventas	Administrador, Vendedor, Cajero (anular requiere credenciales de un Administrador)
Cotizaciones	Administrador, Vendedor
Caja	Administrador, Cajero, Vendedor (catálogo de cajas, incidencias y vales: solo Administrador)
Kardex	Administrador, Bodeguero
Reportes y Usuarios	Administrador

7. Prototipo funcional

7.1 Arquitectura

La aplicación sigue el patrón **Modelo–Vista–Controlador**:

- **Modelos** (models/): única capa que contiene SQL; se conecta a Oracle con python-oracledb en modo thin (sin instalar el cliente de Oracle).
- **Controladores** (controllers/): reciben cada petición, validan los datos y llaman a los modelos. Cada módulo es un Blueprint de Flask.
- **Vistas** (templates/): plantillas HTML con Jinja2 y una hoja de estilos propia (static/css/estilos.css).

Tecnología	Uso
Python 3.12 + Flask	Servidor web y rutas
python-oracledb	Conexión con Oracle, llamadas a procedimientos y parámetros OUT
Jinja2	Plantillas HTML
werkzeug.security	Hash de contraseñas

7.2 Pantallas principales



Figura 8. Inicio de sesión.

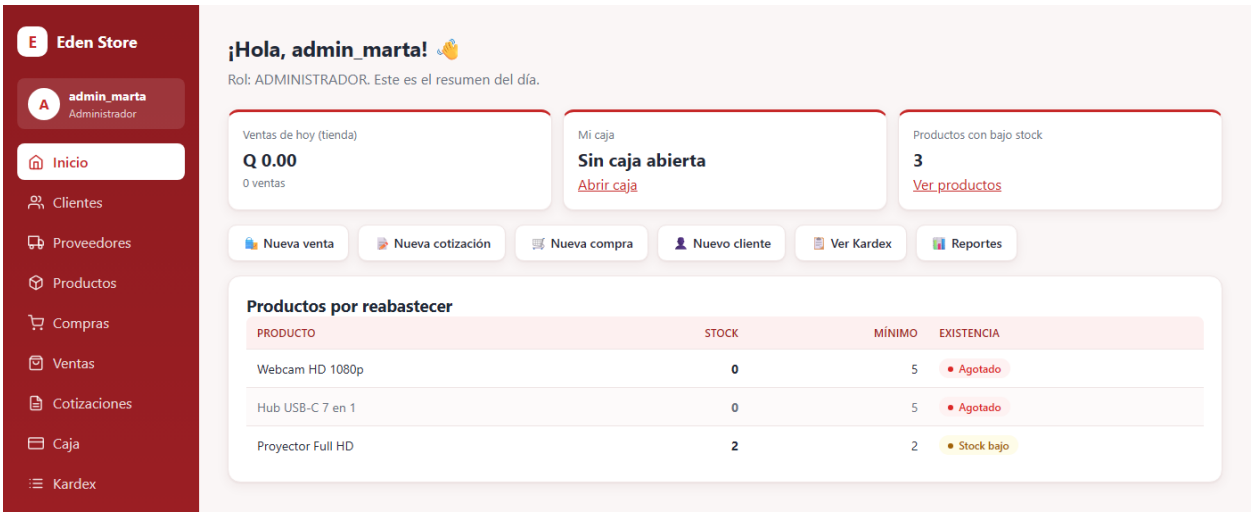


Figura 9. Página de inicio del administrador: ventas del día, estado de su caja y productos por reabastecer.

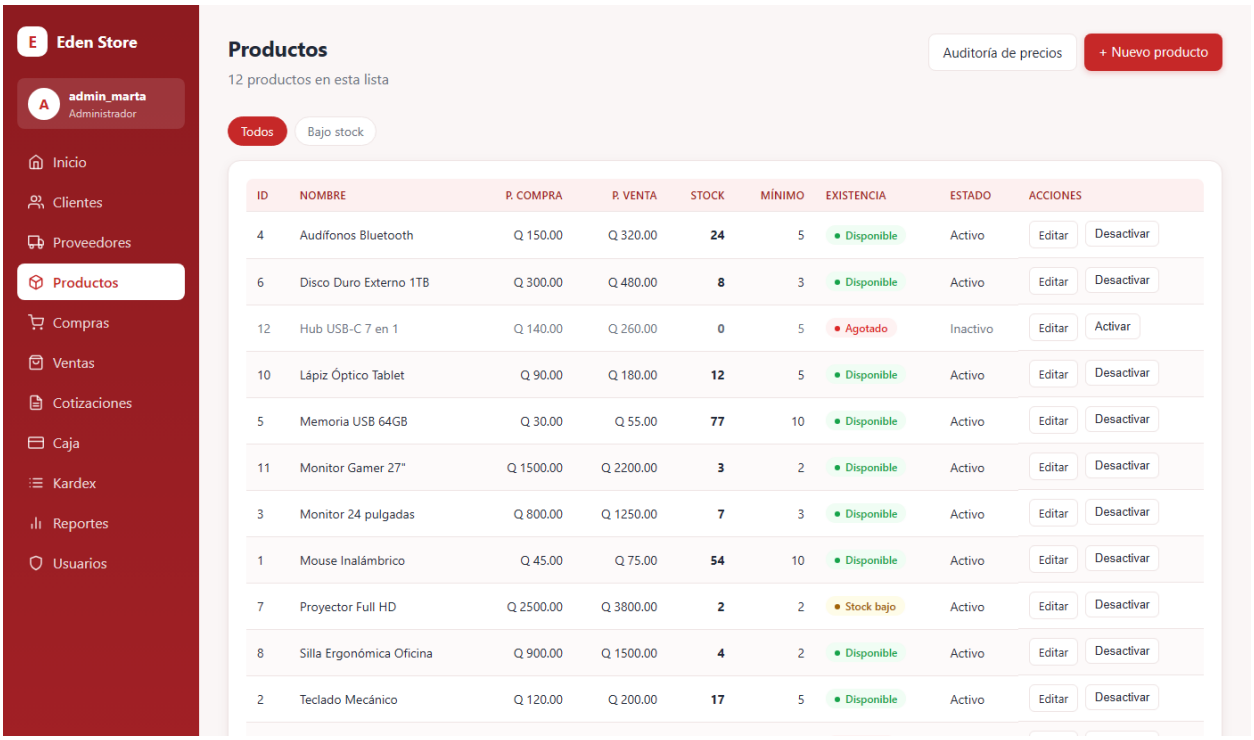


Figura 10. Productos con existencia calculada por FN_STOCK_PRODUCTO y clasificación (disponible, stock bajo, agotado).

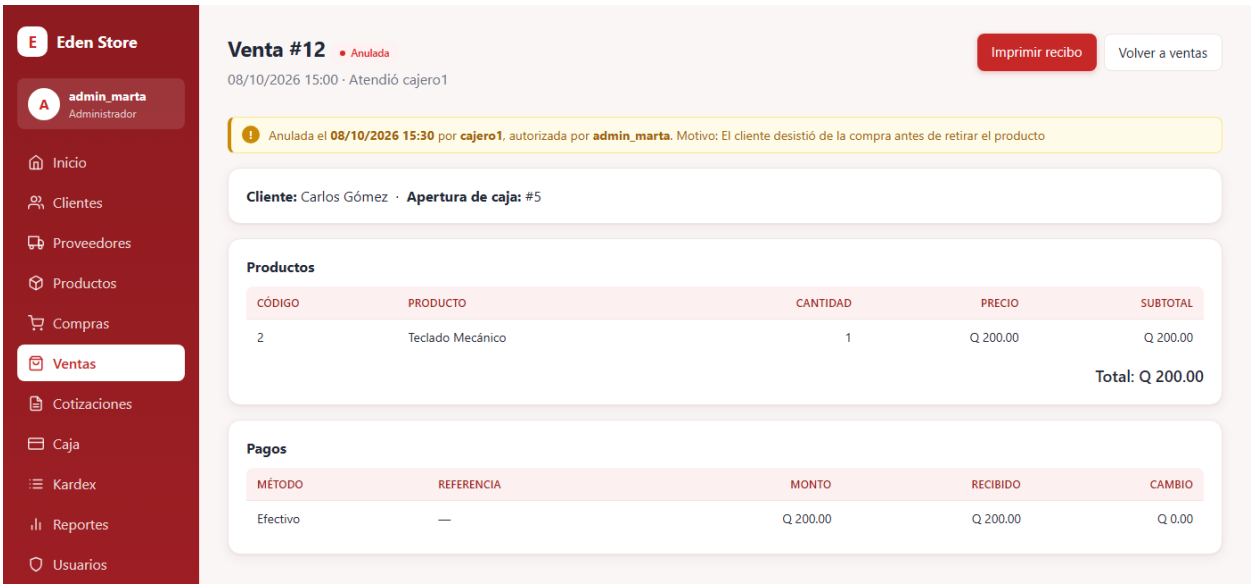


Figura 11. Detalle de una venta anulada: conserva quién la solicitó, quién la autorizó y el motivo.

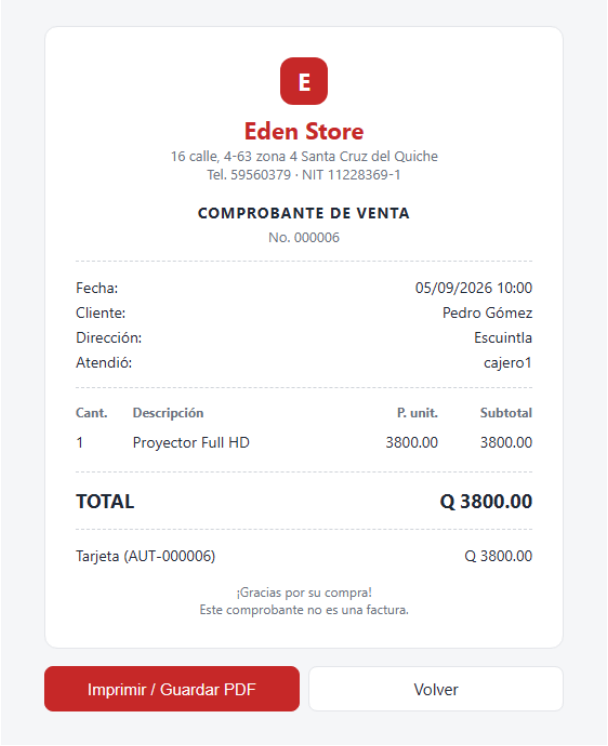


Figura 12. Comprobante de venta imprimible o guardable en PDF.

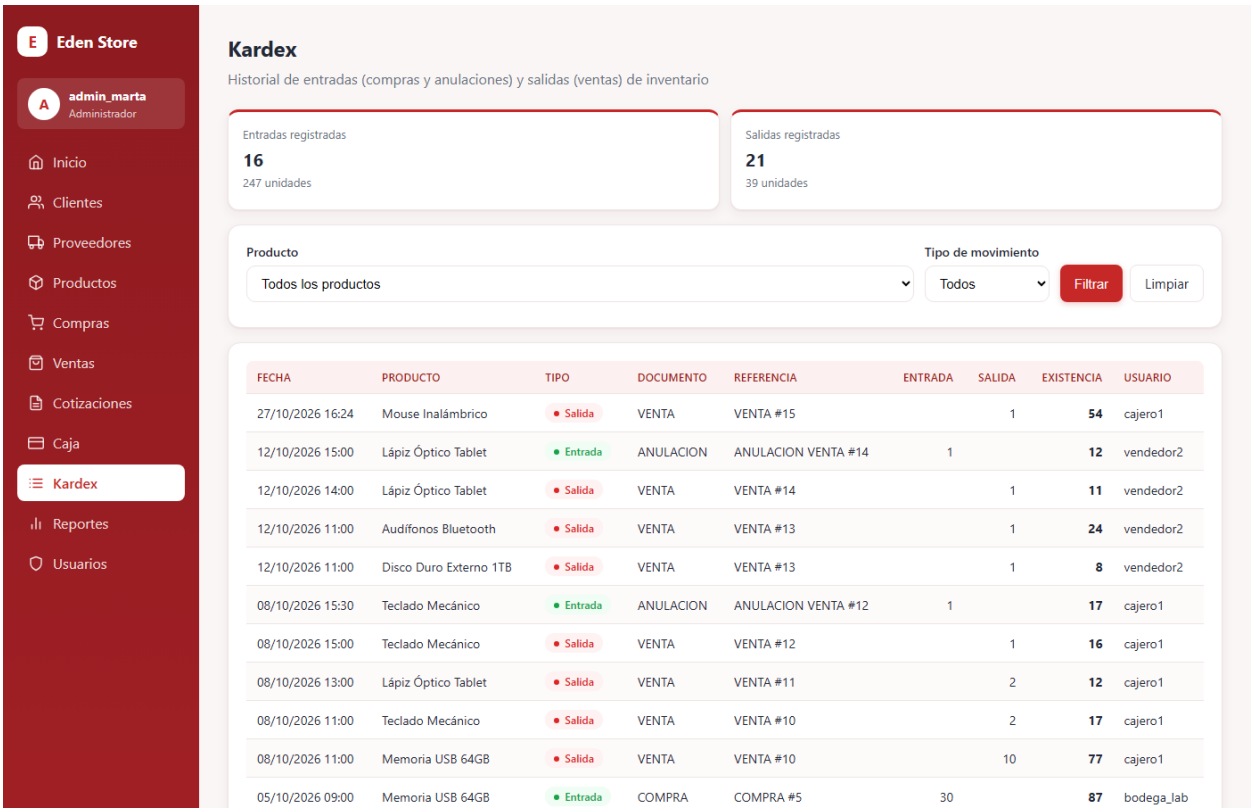


Figura 13. Kardex: entradas (compras y anulaciones) y salidas (ventas) con la existencia resultante.

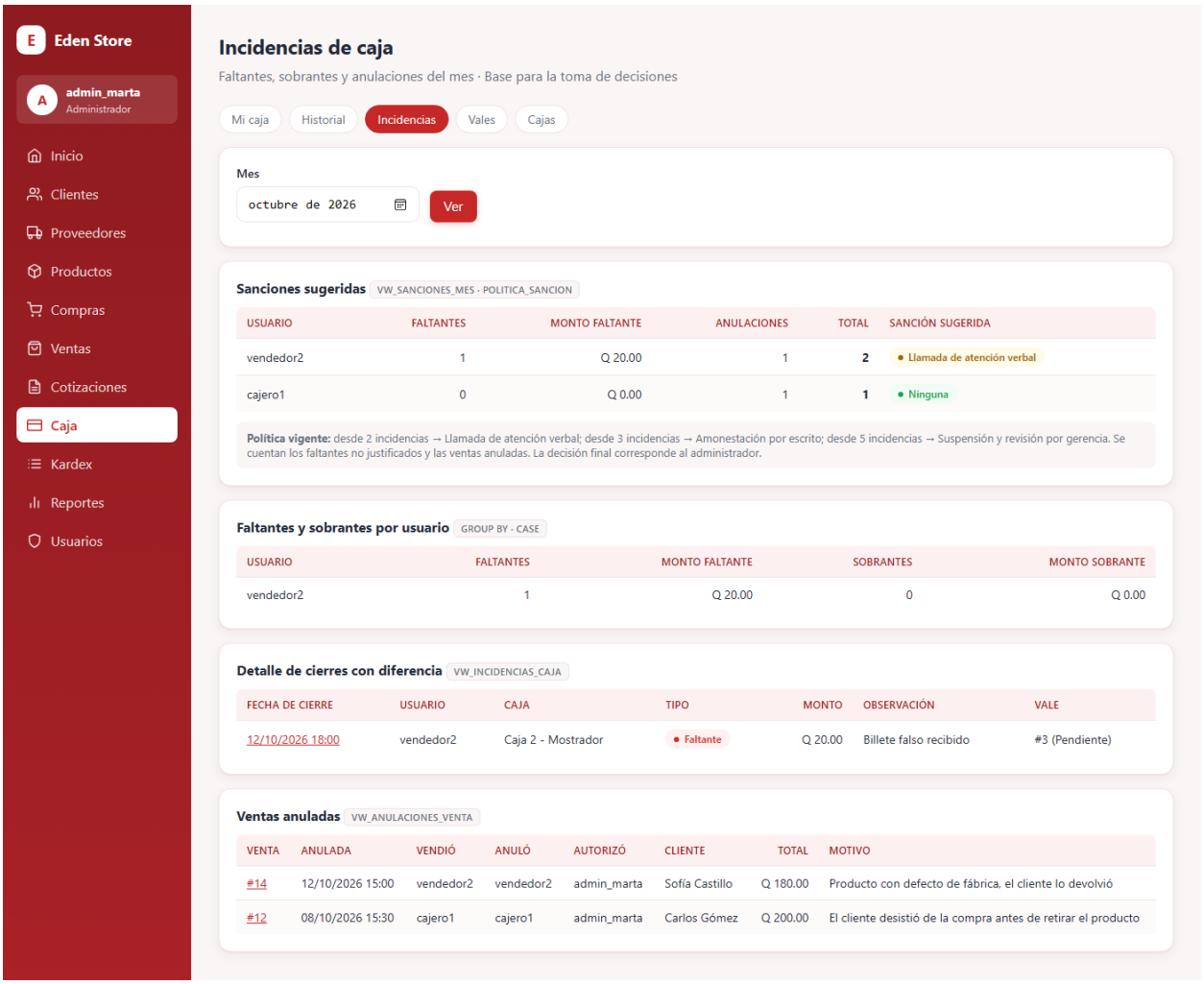


Figura 14. Incidencias de caja: sanciones sugeridas, faltantes, sobrantes y anulaciones del mes.

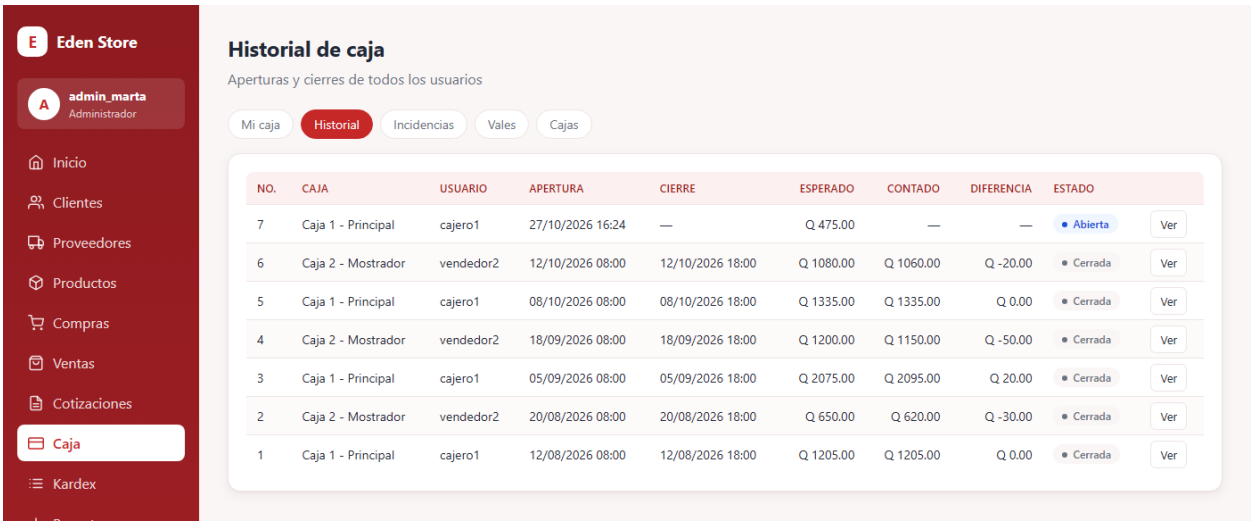


Figura 15. Historial de aperturas y cierres de caja con monto esperado, contado y diferencia.

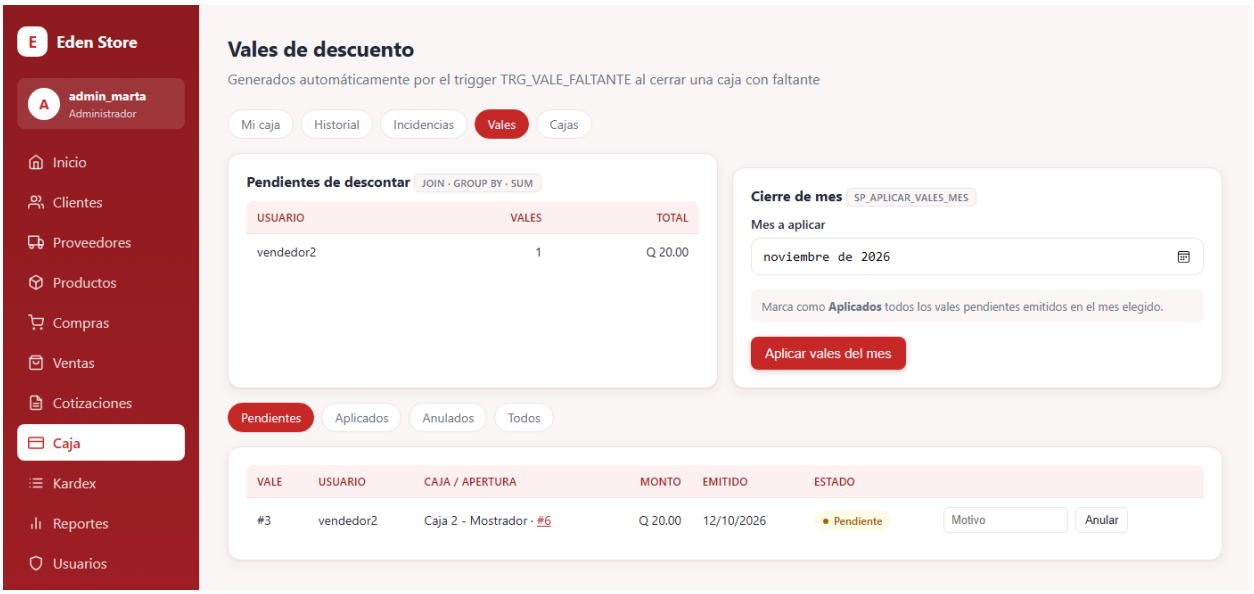


Figura 16. Vales de descuento generados por el trigger TRG_VALE_FALTANTE y proceso de cierre de mes.



Figura 17. Auditoría de precios generada por el trigger TRG_AUDITORIA_PRECIO.



Figura 18. Cotizaciones con estados Abierta, Facturada y Caducada.

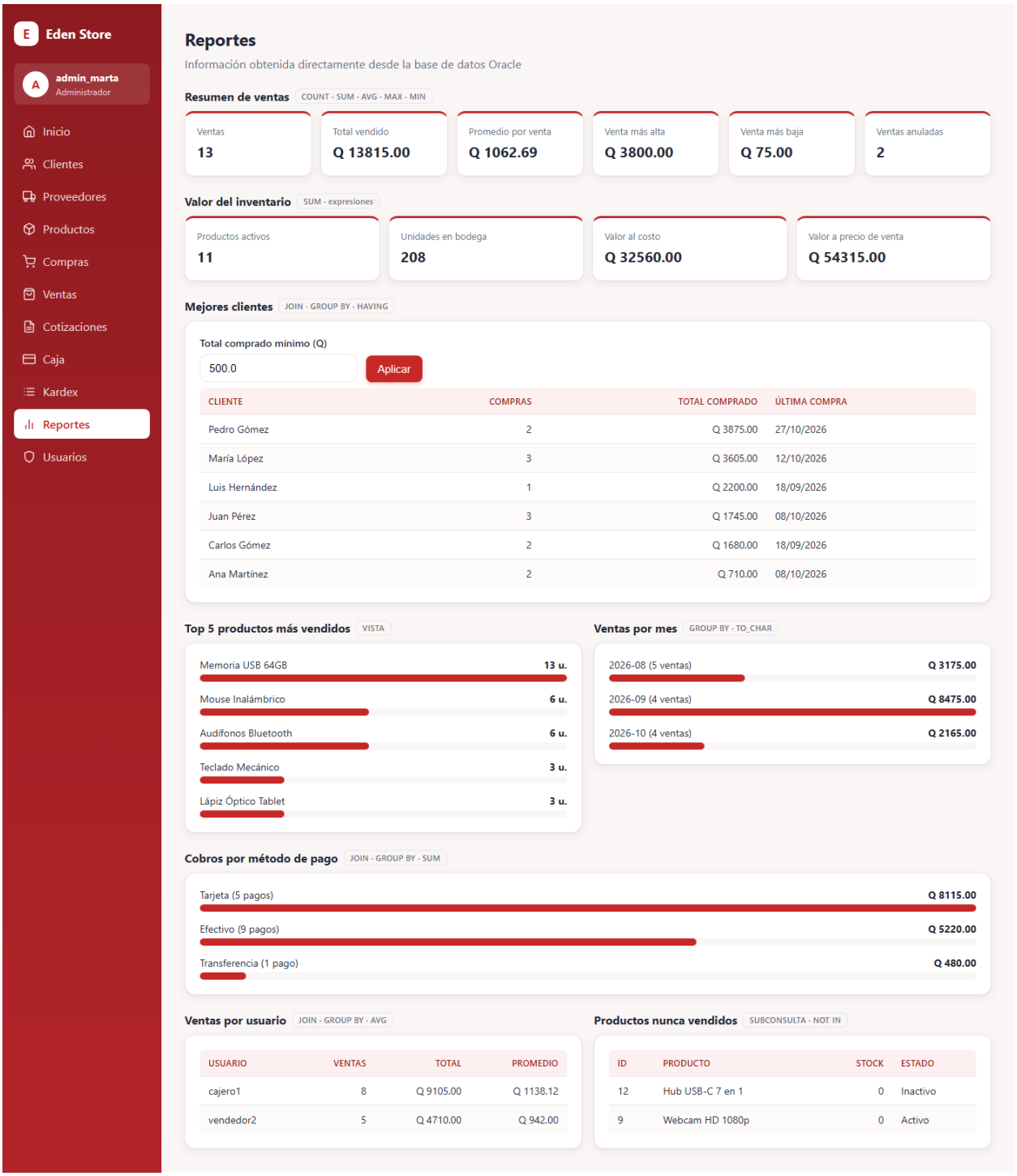


Figura 19. Reportes: indicadores con funciones de agregación, valor del inventario y mejores clientes.

8. Pruebas y validación

8.1 Datos de prueba

La sección 10 del script carga operaciones de **agosto a octubre de 2026**, suficientes para probar todos los módulos y reportes:

Tabla	Registros	Tabla	Registros
USUARIO / ROL / USUARIO_ROL	5 / 4 / 5	VENTAS (2 anuladas)	15
CLIENTES	7	DETALLE_VENTA	21
PROVEEDORES	4	PAGO_VENTA (efectivo, tarjeta, transferencia)	17
PRODUCTOS	12	COTIZACION (abierta, facturada, caducada)	3
COMPRAS / DETALLE_COMPRA	5 / 14	DETALLE_COTIZACION	6
KARDEX	37	CAJA / APERTURA_CAJA	3 / 7
AUDITORIA_PRODUCTO	1	VALE_DESCUENTO (pendiente, aplicado, anulado)	3
METODO_PAGO / POLITICA_SANCION	3 / 3		

8.2 Verificación de la consistencia de los datos

Las consultas 11.2 a 11.5 del script comparan cada dato guardado con su valor calculado. Todas devolvieron **OK** en cada fila:

Consulta	Verifica que…	Resultado
11.2	`El stock de cada producto = entradas − salidas del Kardex.`	12 de 12 productos OK
11.3	El total de cada venta = suma de sus detalles.	15 de 15 ventas OK
11.4	La suma de los pagos = total de cada venta.	15 de 15 ventas OK
11.5	Monto esperado de la caja = monto inicial + efectivo de ventas activas.	7 de 7 aperturas OK

8.3 Pruebas de reglas de negocio y manejo de errores

La sección 12 del script intenta romper cada regla importante. Ninguna prueba deja cambios en la base de datos. Estos son los resultados obtenidos al ejecutarlas:

Prueba	Escenario	Resultado obtenido en Oracle
12.1	Registrar una venta completa y hacer ROLLBACK	Antes: venta #20 y stock 52. Después del ROLLBACK: 0 ventas con ese número y stock 54 (igual que al inicio).
12.2	Vender 100 proyectores cuando hay 2	ORA-20004: Stock insuficiente de Proyector Full HD. Disponible: 2, solicitado: 100.
12.3	Consultar un producto inexistente	ORA-20001: No existe el producto solicitado.
12.4	Vender sin caja abierta	ORA-20013: No existe una apertura de caja activa para este usuario.
12.5	Pagar Q10 de una venta de Q150	ORA-20056: La suma de los pagos (Q10) debe ser igual al total de la venta (Q150).
12.6	Pagar con tarjeta sin número de autorización	ORA-20053: El pago con Tarjeta requiere el número de autorización o referencia.
12.7	Anular sin autorización de un administrador	ORA-20063: La anulación debe ser autorizada por un administrador activo.
12.8	Anular una venta de una caja cerrada	ORA-20062: No se puede anular: la caja de esta venta ya fue cerrada.
12.9	Marcar una venta como anulada sin motivo	ORA-02290: restricción CHK_VENTAS_ANULACION violada.
12.10	Dejar un stock negativo	ORA-02290: restricción CHK_PRODUCTOS_STOCK violada.
12.11	Eliminar un cliente con ventas	ORA-02292: restricción FK_VENTAS_CLIENTE violada - registro secundario encontrado.
12.12	Anular un vale sin motivo	ORA-02290: restricción CHK_VALE_ANULADO violada.
12.13	Repetir el NIT de un proveedor	UNIQUE en acción: ya existe un proveedor con ese NIT (DUP_VAL_ON_INDEX).

Después de ejecutar todas las pruebas, la base conserva 15 ventas, el stock del producto 1 sigue en 54 unidades y no se agregó ningún proveedor: **los datos quedaron consistentes**.

8.4 Validaciones en la aplicación

Antes de llamar a Oracle, los controladores validan los datos obligatorios (campos vacíos, cantidades y montos mayores a cero, correo con @, referencia en pagos con tarjeta o transferencia) para dar mensajes claros. Si una regla solo puede comprobarse en la base de datos (por ejemplo, el stock cuando dos cajeros venden a la vez), el error de Oracle se muestra traducido y la operación se deshace completa.

Además, se recorrieron todas las pantallas con cada rol (administrador, vendedor, cajero y bodeguero) verificando que cada uno vea solo sus módulos y que los datos mostrados coincidan con la base de datos.

9. Instalación, ejecución y respaldo

9.1 Base de datos

1. Conectado como **SYSDBA** en XEPDB1, crear el usuario SISTEMA_VENTAS y darle sus privilegios (Anexo A, paso 1).
2. Conectado como **SISTEMA_VENTAS**, abrir database/proyecto_sistema_ventas.sql en SQL Developer y ejecutarlo completo con F5.
3. Conectado como **SYSDBA**, ejecutar los roles, los GRANT y los usuarios de base de datos (Anexo A, pasos 2 a 4).

9.2 Aplicación

1. Copiar config.example.py como config.py y escribir los datos de conexión.
2. Crear el entorno virtual: python -m venv venv y activarlo con .\venv\Scripts\Activate.ps1.
3. Instalar las dependencias: pip install -r requirements.txt.
4. Asignar las contraseñas cifradas a los usuarios de prueba: python establecer_passwords.py.
5. Iniciar la aplicación con python app.py y abrir http://127.0.0.1:5000.

9.3 Respaldo y restauración

El respaldo se hace con **Oracle Data Pump** usando el directorio DIR_RESPALDOS (C:\respaldos_oracle):

```
-- Respaldo del esquema completo (CMD)
expdp sistema_ventas@localhost:1521/xepdb1 SCHEMAS=sistema_ventas
DIRECTORY=DIR_RESPALDOS
    DUMPFILE=sistema_ventas.dmp LOGFILE=exp_sistema_ventas.log

-- Restauración de prueba en otro esquema (CMD)
impdp system@localhost:1521/xepdb1 DIRECTORY=DIR_RESPALDOS
DUMPFILE=sistema_ventas.dmp
    LOGFILE=imp_sistema_ventas_rest.log
REMAP_SCHEMA=sistema_ventas:sistema_ventas_rest
EXCLUDE=PROCOBJ
```

Durante el desarrollo se restauró el esquema completo desde un respaldo: se importaron las 20 tablas con sus datos, y todos los objetos (vistas, procedimientos, funciones, triggers e índices) quedaron válidos, con el job programado activo.

10. Conclusiones y recomendaciones

10.1 Conclusiones

- La normalización hasta 3FN eliminó la repetición de datos de clientes, productos y usuarios que existía en las hojas de cálculo; las únicas redundancias que quedan son controladas y se verifican con consultas.
- Colocar las reglas críticas en Oracle (restricciones, procedimientos y triggers) garantiza la integridad aunque alguien acceda a la base sin pasar por la aplicación.
- Las transacciones con COMMIT y ROLLBACK aseguran que una venta, una compra o una anulación se registren completas o no se registren, como demostró la prueba 12.1.
- El Kardex y la auditoría de precios dan trazabilidad completa: se sabe quién, cuándo y por qué cambió cada existencia o precio.
- El control de caja con vales, incidencias y sanciones sugeridas convierte los datos en información útil para que el administrador tome decisiones.
- La seguridad en dos niveles (roles de Oracle y permisos en la aplicación) aplica el principio de menor privilegio.

10.2 Recomendaciones y mejoras futuras

- Integrar la facturación electrónica FEL de la SAT.
- Agregar notas de crédito y devoluciones parciales para ventas de cajas ya cerradas.
- Manejar ventas al crédito y cuentas por cobrar.
- Soportar varias sucursales o bodegas con transferencias de inventario.
- Programar respaldos automáticos diarios y guardarlos fuera del servidor.
- Desplegar la aplicación en un servidor Ubuntu con HTTPS.

Anexo A. Diccionario de datos

Generado a partir de las sentencias CREATE TABLE del script. Abreviaturas: **PK** llave primaria, **FK** llave foránea, **UK** valor único, **NN** obligatorio (NOT NULL).

ROL

Perfiles de acceso de la aplicación.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_rol	NUMBER (10)	NN		PK	
nombre_rol	VARCHAR2(50)	NN		UK	
descripcion	VARCHAR2(150)				

USUARIO

Usuarios que inician sesión en la aplicación.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_usuario	NUMBER (10)	NN		PK	
usuario	VARCHAR2(50)	NN		UK	
password	VARCHAR2(255)	NN			hash (scrypt), nunca texto plano
estado	CHAR(1)	NN	'A'		

Restricciones CHECK: chk_usuario_estado.

USUARIO_ROL

Tabla asociativa entre USUARIO y ROL (M:N).

Columna	Tipo	Nulo	Default	Llaves	Observación
id_usuario	NUMBER (10)	NN		PK, FK → USUARIO	

Columna	Tipo	Nulo	Default	Llaves	Observación
id_rol	NUMBER (10)	NN		PK, FK → ROL	

CLIENTES

Clientes de la tienda.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_cliente	NUMBER (10)	NN		PK	
nombre	VARCHAR2(100)	NN			
direccion	VARCHAR2(150)	NN			
telefono	VARCHAR2(20)	NN			
correo	VARCHAR2(150)			UK	

PROVEEDORES

Proveedores de productos.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_proveedor	NUMBER (10)	NN		PK	
nombre	VARCHAR2(100)	NN			
direccion	VARCHAR2(150)	NN			
telefono	VARCHAR2(20)	NN			
nit	VARCHAR2(20)	NN		UK	

PRODUCTOS

Catálogo de productos con precios y existencias.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_producto	NUMBER (10)	NN		PK	
nombre	VARCHAR2(100)	NN			
precio_compra	NUMBER (12,2)	NN			
precio_venta	NUMBER (12,2)	NN			
stock	NUMBER (8)	NN	0		
estado	CHAR(1)	NN	'A'		
fecha_registro	DATE	NN	SYSDATE		
stock_minimo	NUMBER (10)	NN	5		

Restricciones CHECK: chk_productos_estado, chk_productos_stock, chk_productos_stock_min, chk_productos_precios.

CAJA

Cajas físicas del negocio.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_caja	NUMBER (10)	NN		PK	
nombre	VARCHAR2(50)	NN			
estado	CHAR(1)	NN	'A'		

Restricciones CHECK: chk_caja_est.

APERTURA_CAJA

Aperturas y cierres de caja por usuario.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_apertura	NUMBER (10)	NN		PK	

Columna	Tipo	Nulo	Default	Llaves	Observación
id_caja	NUMBER (10)	NN		FK → CAJA	
id_usuario	NUMBER (10)	NN		FK → USUARIO	
fecha_apertura	DATE	NN	SYSDATE		
monto_inicial	NUMBER (12,2)	NN			
fecha_cierre	DATE				
monto_esperado	NUMBER (12,2)		0		
monto_contado	NUMBER (12,2)		0		
diferencia	NUMBER (12,2)		0		
estado	CHAR(1)	NN	'A'		
observacion_cierre	VARCHAR2(200)				

Restricciones CHECK: chk_ape_est, chk_ape_monto_ini, chk_ape_cierre.

COTIZACION

Encabezado de cotizaciones (A=Abierta, C=Caducada, F=Facturada).

Columna	Tipo	Nulo	Default	Llaves	Observación
id_cotizacion	NUMBER (10)	NN		PK	
id_cliente	NUMBER (10)	NN		FK → CLIENTES	
id_usuario	NUMBER (10)	NN		FK → USUARIO	
fecha	DATE	NN	SYSDATE		

Columna	Tipo	Nulo	Default	Llaves	Observación
fecha_vencimiento	DATE	NN			
total	NUMBER(12,2)	NN	0		
estado	CHAR(1)	NN	'A'		A=Abierta, C=Caducada, F=Facturada

Restricciones CHECK: chk_cotiz_est, chk_cotiz_total.

DETALLE_COTIZACION

Productos de cada cotización.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_detalle_cotizacion	NUMBER(10)	NN		PK	
id_cotizacion	NUMBER(10)	NN		FK → COTIZACION	
id_producto	NUMBER(10)	NN		FK → PRODUCTOS	
cantidad	NUMBER(8)	NN			
precio	NUMBER(12,2)	NN			
subtotal	NUMBER(12,2)	NN			

Restricciones CHECK: chk_detcotiz_cantidad, chk_detcotiz_precio.

COMPRAS

Encabezado de compras a proveedores.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_compra	NUMBER(10)	NN		PK	

Columna	Tipo	Nulo	Default	Llaves	Observación
fecha	DATE	NN	SYSDATE		
total	NUMBER (12,2)	NN	0		
id_proveedor	NUMBER (10)	NN		FK → PROVEEDORES	
id_usuario	NUMBER (10)	NN		FK → USUARIO	

Restricciones CHECK: chk_compras_total.

DETALLE_COMPRA

Productos de cada compra.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_detalle_compra	NUMBER (10)	NN		PK	
id_compra	NUMBER (10)	NN		FK → COMPRAS	
id_producto	NUMBER (10)	NN		FK → PRODUCTOS	
cantidad	NUMBER (8)	NN			
precio_costo	NUMBER (12,2)	NN			
subtotal	NUMBER (12,2)	NN			

Restricciones CHECK: chk_detcompra_cantidad, chk_detcompra_precio.

VENTAS

Encabezado de ventas (A=Activa, N=Anulada).

Columna	Tipo	Nulo	Default	Llaves	Observación
id_venta	NUMBER (10)	NN		PK	
fecha	DATE	NN	SYSDATE		
total	NUMBER (12,2)	NN	0		
id_cliente	NUMBER (10)	NN		FK → CLIENTES	
id_usuario	NUMBER (10)	NN		FK → USUARIO	
id_apertura	NUMBER (10)	NN		FK → APERTURA_CAJA	toda venta pertenece a una apertura de caja
id_cotizacion	NUMBER (10)			FK → COTIZACION, UK	opcional: venta generada desde una cotización
estado	CHAR(1)	NN	'A'		A=Activa, N=Anulada
fecha_anulacion	DATE				
id_usuario_anula	NUMBER (10)			FK → USUARIO	quien solicitó la anulación
id_usuario_autoriza	NUMBER (10)			FK → USUARIO	administrador que la autorizó
motivo_anulacion	VARCHAR2(200)				

Restricciones CHECK: chk_ventas_total, chk_ventas_estado, chk_ventas_anulacion.

DETALLE_VENTA

Productos de cada venta.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_detalle_venta	NUMBER (10)	NN		PK	
id_venta	NUMBER (10)	NN		FK → VENTAS	
id_producto	NUMBER (10)	NN		FK → PRODUCTOS	
cantidad	NUMBER (8)	NN			
precio_venta	NUMBER (12,2)	NN			
subtotal	NUMBER (12,2)	NN			

Restricciones CHECK: chk_detventa_cantidad, chk_detventa_precio.

KARDEX

Movimientos de inventario (ENTRADA / SALIDA).

Columna	Tipo	Nulo	Default	Llaves	Observación
id_kardex	NUMBER (10)	NN		PK	
id_producto	NUMBER (10)	NN		FK → PRODUCTOS	

Columna	Tipo	Nulo	Default	Llaves	Observación
fecha	DATE	NN	SYSDATE		
tipo_movimiento	VARCHAR2(10)	NN			
tipo_documento	VARCHAR2(20)	NN			
referencia	VARCHAR2(50)	NN			
cantidad_entrada	NUMBER(8)	NN	0		
cantidad_salida	NUMBER(8)	NN	0		
existencia_final	NUMBER(8)	NN			
id_usuario	NUMBER(10)	NN		FK → USUARIO	

Restricciones CHECK: chk_kardex_tipo, chk_kardex_cantidad, chk_kardex_existencia, chk_kardex_coherencia.

AUDITORIA_PRODUCTO

Bitácora de cambios de precio de venta.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_auditoria	NUMBER(10)	NN		PK	
id_producto	NUMBER(10)	NN		FK → PRODUCTOS	
precio_anterior	NUMBER(12,2)				
precio_nuevo	NUMBER(12,2)				

Columna	Tipo	Nulo	Default	Llaves	Observación
usuario	VARCHAR2(50)	NN			
fecha	DATE	NN	SYSDATE		
tipo_operacion	VARCHAR2(20)	NN			

VALE_DESCUENTO

Vales por faltantes de caja (P=Pendiente, A=Aplicado, N=Anulado).

Columna	Tipo	Nulo	Default	Llaves	Observación
id_vale	NUMBER(10)	NN		PK	
id_apertura	NUMBER(10)	NN		FK → APERTURA_CAJA, UK	
monto	NUMBER(12,2)	NN			
estado	CHAR(1)	NN	'P'		P=Pendiente, A=Aplicado, N=Anulado
fecha_emision	DATE	NN	SYSDATE		
fecha_aplicacion	DATE				
motivo_anulacion	VARCHAR2(200)				

Restricciones CHECK: chk_vale_monto, chk_vale_estado, chk_vale_aplicado, chk_vale_anulado.

METODO_PAGO

Catálogo de métodos de pago.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_metodo	NUMBER(10)	NN		PK	
nombre	VARCHAR2(30)	NN		UK	
afecta_caja	CHAR(1)	NN			S: el dinero entra a la gaveta (efectivo)
requiere_referencia	CHAR(1)	NN			S: exige número de autorización o boleta
estado	CHAR(1)	NN	'A'		

Restricciones CHECK: chk_metodo_caja, chk_metodo_referencia, chk_metodo_estado.

PAGO_VENTA

Pagos registrados en cada venta.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_pago	NUMBER(10)	NN		PK	
id_venta	NUMBER(10)	NN		FK → VENTAS	
id_metodo	NUMBER(10)	NN		FK → METODO_PAGO	
monto	NUMBER(12,2)	NN			
referencia	VARCHAR2(50)				autorización de tarjeta o número de boleta

Columna	Tipo	Nulo	Default	Llaves	Observación
monto_recibido	NUMBER (12,2)				solo efectivo: con cuánto pagó el cliente

Restricciones CHECK: chk_pago_monto, chk_pago_recibido.

POLITICA_SANCION

Sanción sugerida según el número de incidencias del mes.

Columna	Tipo	Nulo	Default	Llaves	Observación
id_politica	NUMBER (10)	NN		PK	
minimo_incidencias	NUMBER (3)	NN		UK	
sancion	VARCHAR2(100)	NN			

Restricciones CHECK: chk_politica_minimo.